OmniCode: Chapter 8: Track 1 - Diagnostics & 4-Block Linter

Spatial Ruler, Morphological Bounds & Block Validation

CHAPTER IDENTIFIER
CH 08

NOTEBOOKLM WORD COUNT
14,643 words

CATALOGED CHAPTER FILES
28 files (3,691 lines)

GENERATION DATE
2026-10-16 20:35:58

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How does the 4-block linter enforce spatial line geometry?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (28 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/audit.rs**	RS	93	345	3.0
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/mod.rs**	RS	396	1,287	15.4
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/detector.rs**	RS	241	919	8.9
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/mod.rs**	RS	61	209	2.0
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/omni.rs**	RS	175	680	7.9
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/tape.rs**	RS	230	849	10.4
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/mod.rs**	RS	84	265	3.1
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/reel.rs**	RS	106	508	4.6
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/score.rs**	RS	71	267	2.5
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/tape_setup.rs**	RS	112	456	4.4
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/mod.rs**	RS	84	264	3.0
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/reel.rs**	RS	115	508	4.8
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/score.rs**	RS	71	266	2.5
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/tape_rolls.rs**	RS	116	512	4.6
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/assembly.rs**	RS	133	560	6.6
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/mod.rs**	RS	19	63	0.7
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/mod.rs**	RS	84	272	3.1
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/reel.rs**	RS	113	551	5.1
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/score.rs**	RS	71	271	2.6
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/tape_commit.rs**	RS	112	499	4.6
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/grammar.rs**	RS	142	589	6.3
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/mod.rs**	RS	34	144	1.5
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/spatial.rs**	RS	109	443	4.8
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/head.rs**	RS	247	953	10.3
25	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/mod.rs**	RS	112	388	4.8
26	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/reel.rs**	RS	250	995	10.6
27	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/score.rs**	RS	104	452	4.2
28	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/scaffold/mod.rs**	RS	206	1,128	7.8

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/audit.rs RS 93 LINES 345 WORDS 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal 4-Block File Audit Score & QuadState Gate Taxonomy )
// ============================================================================
//! Universal 4-Block File Audit Score & QuadState Compilation Gate
//!
//! Synthesizes the four block-level invariant scores into a single composite
//! metric on the continuous balanced scale $[-100 \dots +100]$, and maps the result
//! to an operational QuadState compilation gate:
//! - `+1 : PROCEED` (Composite >= +70): Eligible to compile and lower to machine words.
//! - ` 0 : AWAIT`   (Composite +50..+70): Ambient equilibrium hold pending advisories.
//! - `-1 : ROLLBACK` (Composite +20..+50): Precondition/covenant deficit; causal tape rolls back.
//! - `HALT : NULL`  (Composite < +20): Ineligible to compile; execution stops.

use super::metadata::MetadataScore;
use super::opening::OpeningScore;
use super::body::BodyScore;
use super::closing::ClosingScore;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum QuadStateGate {
    Proceed,
    Await,
    Rollback,
    Halt,
}

impl QuadStateGate {
    pub fn name(&self) -> &'static str {
        match self {
            QuadStateGate::Proceed => "+1 : PROCEED (Eligible to Compile)",
            QuadStateGate::Await => " 0 : AWAIT (Advisory Equilibrium Hold)",
            QuadStateGate::Rollback => "-1 : ROLLBACK (Invariant Reversion)",
            QuadStateGate::Halt => "HALT : NULL (Compilation Disqualified)",
        }
    }

    pub fn is_proceed(&self) -> bool {
        matches!(self, QuadStateGate::Proceed)
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FileAuditScore {
    pub block1_metadata: MetadataScore,
    pub block2_opening: OpeningScore,
    pub block3_body: BodyScore,
    pub block4_closing: ClosingScore,
    pub composite_score: i32,
    pub gate: QuadStateGate,
    pub eligible_to_compile: bool,
}

impl FileAuditScore {
    /// Computes the composite 4-Block audit score and resolves the QuadState gate.
    pub fn calculate(
        block1: MetadataScore,
        block2: OpeningScore,
        block3: BodyScore,
        block4: ClosingScore,
        has_fatal_errors: bool,
    ) -> Self {
        let b1 = block1.total as i32;
        let b2 = block2.total;
        let b3 = block3.total;
        let b4 = block4.total;

        let composite = ((b1 + b2 + b3 + b4) / 4).clamp(-100, 100);

        let gate = if has_fatal_errors || composite < 20 {
            QuadStateGate::Halt
        } else if composite >= 70 {
            QuadStateGate::Proceed
        } else if composite >= 50 {
```

```
            QuadStateGate::Await
        } else {
            QuadStateGate::Rollback
        };

        let eligible_to_compile = gate.is_proceed() && !has_fatal_errors;

        Self {
            block1_metadata: block1,
            block2_opening: block2,
            block3_body: block3,
            block4_closing: block4,
            composite_score: composite,
            gate,
            eligible_to_compile,
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/mod.rs RS 396 LINES 1287 WORDS 15.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniLinter Coordinator, Subsystem Exports & Invariant Architecture )
// ============================================================================
//! OmniCode Diagnostics: Polymorphic Structural, Grammar & Spatial Linter
//!
//! Enforces the structural invariants, lexical syntax, grammar ASTs, and physical
//! spatial geometry across all OmniCode compilation units and execution ribbons:
//! 1. `.omni` (Source Reel): Canonical 4-Block Architecture (Metadata, Opening, Body, Closing).
//! 2. `.tape` (Process Ribbon): Sequential Tape Head, Cardinal Coordinates, Rolls Stream & Commit Status.
//! 3. Spatial Geometry: Strict 80-column ruler for dual side-panel IDE viewports.
//! 4. Grammar & Syntax: SVO verbs, state-prime synchronization, Living Asides, and boundary physics.
//! 5. Assembly Pre-flight: Partition bounds, silicon MMIO targets, and register verification.

use std::path::Path;

use crate::core::filetype::OmniFileType;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};

pub mod audit;
pub mod body;
pub mod closing;
pub mod metadata;
pub mod opening;
pub mod preflight;
pub mod scaffold;
pub mod structure;
pub mod syntax;

pub use audit::{FileAuditScore, QuadStateGate};
pub use body::{BodyLinter, BodyScore, ReelBodyLinter, TapeRollsLinter};
pub use closing::{ClosingLinter, ClosingScore, ReelClosingLinter, TapeCommitLinter};
pub use metadata::{MetadataLinter, MetadataScore, ReelMetadataLinter, TapeHeadLinter};
pub use opening::{OpeningLinter, OpeningScore, ReelOpeningLinter, TapeSetupLinter};
pub use preflight::AssemblyLinter;
pub use scaffold::LinterScaffolder;
pub use structure::{BlockDetector, BlockKind, BlockSlice, DetectedBlock, FileBlockSlices, OmniReelLinter, TapeLinter};
pub use syntax::{GrammarLinter, SpatialLintConfig, SpatialLinter};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( LintReport & OmniLinter Initialization )
// ============================================================================

#[derive(Debug, Clone)]
pub struct LintReport {
    pub file_path: String,
    pub file_type: Option<OmniFileType>,
    pub blocks_found: Vec<DetectedBlock>,
    pub metadata_score: Option<MetadataScore>,
    pub audit_score: Option<FileAuditScore>,
    pub is_valid: bool,
    pub errors: Vec<String>,
    pub warnings: Vec<String>,
}

pub struct OmniLinter {
    pub logger: DiagnosticLogger,
    pub debugger: OmniDebugger,
    pub config: SpatialLintConfig,
}

impl OmniLinter {
    /// Creates a new linter bound to a diagnostic logger with default spatial settings.
    pub fn new(logger: DiagnosticLogger) -> Self {
        let debugger = OmniDebugger::new(logger.clone(), 100);
        Self {
            logger,
            debugger,
            config: SpatialLintConfig::default(),
        }
    }

```

```rust
    /// Creates a new linter with custom debugger and spatial configuration.
    pub fn with_debugger(logger: DiagnosticLogger, debugger: OmniDebugger) -> Self {
        Self {
            logger,
            debugger,
            config: SpatialLintConfig::default(),
        }
    }

    /// Configures strict 80-column ruler enforcement.
    pub fn with_strict_geometry(mut self, strict: bool) -> Self {
        self.config.strict_column_limit = strict;
        self
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Linting Pipelines: Polymorphic Dispatch & Assembly Pre-flight )
// ============================================================================

impl OmniLinter {
    /// Lints a source string across structural, grammar, and spatial geometry invariants.
    pub fn lint_source(&self, file_path: &str, content: &str) -> LintReport {
        let file_type = OmniFileType::from_path(file_path);

        let mut report = LintReport {
            file_path: file_path.to_string(),
            file_type,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        // Pass 1: Spatial Geometry & Layout Ruler (<= 80 columns)
        SpatialLinter::lint(content, &mut report, &self.config);

        // Pass 2: Delimiter & Assembly Pre-flight Checks
        AssemblyLinter::preflight(content, &mut report);

        // Pass 3: Lexical Syntax & Grammar AST Verification
        GrammarLinter::lint(
            file_path,
            content,
            self.logger.clone(),
            self.debugger.clone(),
            &mut report,
        );

        // Pass 4: Polymorphic Structural Architecture Dispatch
        let is_tape = match file_type {
            Some(OmniFileType::Tape) => true,
            _ => {
                let trimmed_start = content.trim_start();
                trimmed_start.starts_with("= [Tape")
                    || trimmed_start.starts_with("[Tape.")
                    || (trimmed_start.contains("[ ROLL") && !trimmed_start.contains("BLOCK 1"))
            }
        };

        if is_tape {
            TapeLinter::lint(file_path, content, &mut report);
        } else {
            OmniReelLinter::lint(file_path, content, &mut report);
        }

        // Pass 5: Block 1 Invariant & Metadata Scoring
        let meta_score = MetadataLinter::evaluate(file_path, content, is_tape, &self.config, &mut report);
        report.metadata_score = Some(meta_score.clone());

        // Pass 6: Full 4-Block Invariant Audit & QuadState Gate Synthesis
        let open_score = OpeningLinter::evaluate(file_path, content, is_tape, &self.config);
        let body_score = BodyLinter::evaluate(file_path, content, is_tape, &self.config);
        let close_score = ClosingLinter::evaluate(file_path, content, is_tape, &self.config);

```

```
        report.is_valid = report.errors.is_empty();

        let audit = FileAuditScore::calculate(
            meta_score,
            open_score,
            body_score,
            close_score,
            !report.is_valid,
        );
        report.audit_score = Some(audit);

        report.is_valid = report.errors.is_empty();

        if !report.is_valid {
            for err in &report.errors {
                self.logger.failpoint(
                    FailpointType::SyntaxAnomaly,
                    "LINTER",
                    format!("[{file_path}] {err}"),
                );
            }
        } else {
            self.logger.log(
                "LINTER",
                format!("File [{file_path}] satisfies structural, grammar, and spatial invariants."),
            );
        }

        report
    }

    /// Lints a file directly on disk.
    pub fn lint_file(&self, path: impl AsRef<Path>) -> Result<LintReport, String> {
        let p = path.as_ref();
        let content = std::fs::read_to_string(p)
            .map_err(|e| format!("Failed to read file {}: {e}", p.display()))?;
        Ok(self.lint_source(&p.display().to_string(), &content))
    }

    /// Pre-flight gate for the Assembly pipeline: returns Ok only if zero fatal errors exist.
    pub fn lint_for_assembly(&self, file_path: &str, content: &str) -> Result<LintReport, String> {
        let report = self.lint_source(file_path, content);
        if !report.is_valid {
            Err(format!(
                "Pre-assembly linting failed for '{}' with {} errors: {}",
                file_path,
                report.errors.len(),
                report.errors.join("; ")
            ))
        } else {
            Ok(report)
        }
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for an episodic cognitive scroll (.scroll).
    pub fn auto_scaffold_scroll(session_id: &str, prompt: &str) -> String {
        LinterScaffolder::auto_scaffold_scroll(session_id, prompt)
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for a given module name (.omni).
    pub fn auto_scaffold_omni(module_name: &str, intent_aside: &str) -> String {
        LinterScaffolder::auto_scaffold_omni(module_name, intent_aside)
    }

    /// Auto-scaffolds a compliant `.tape` process ribbon skeleton.
    pub fn auto_scaffold_tape(tape_name: &str, intent_aside: &str) -> String {
        LinterScaffolder::auto_scaffold_tape(tape_name, intent_aside)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Comprehensive Unit Tests & Subsystem Sealing )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
```

```
231
232     #[test]
233     fn test_linter_character_omni_source_reel_passes() {
234         let logger = DiagnosticLogger::new(false);
235         let linter = OmniLinter::new(logger);
236         let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
237         if path.exists() {
238             let report = linter.lint_file(&path).expect("Linting character.omni");
239             assert!(report.is_valid, "character.omni must pass linter with 0 errors: {:?}", report.errors);
240             assert_eq!(report.blocks_found.len(), 4, "Must find all 4 blocks in character.omni");
241             let audit = report.audit_score.as_ref().expect("Audit score must be computed");
242             assert!(audit.eligible_to_compile, "character.omni must be eligible to compile: {:?}", audit.gate);
243             assert_eq!(audit.gate, QuadStateGate::Proceed);
244             assert_eq!(audit.composite_score, 100, "character.omni must achieve composite score 100");
245         }
246     }
247
248     #[test]
249     fn test_linter_character_tape_process_ribbon_passes() {
250         let logger = DiagnosticLogger::new(false);
251         let linter = OmniLinter::new(logger);
252         let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
253         if path.exists() {
254             let report = linter.lint_file(&path).expect("Linting character.tape");
255             assert!(report.is_valid, "character.tape must pass linter with 0 errors: {:?}", report.errors);
256             assert!(!report.blocks_found.is_empty(), "Must detect blocks/head in character.tape");
257             let audit = report.audit_score.as_ref().expect("Audit score must be computed");
258             assert!(audit.eligible_to_compile, "character.tape must be eligible to compile: {:?}", audit.gate);
259             assert_eq!(audit.gate, QuadStateGate::Proceed);
260             assert_eq!(audit.composite_score, 100, "character.tape must achieve composite score 100");
261         }
262     }
263
264     #[test]
265     fn test_linter_number_omni_source_reel_passes() {
266         let logger = DiagnosticLogger::new(false);
267         let linter = OmniLinter::new(logger);
268         let path = crate::core::filetype::resolve_track_path(
269             "lexicon/01_nouns/ideas/types/number.omni",
270         );
271         if path.exists() {
272             let report = linter.lint_file(&path).expect("Linting number.omni");
273             assert!(
274                 report.is_valid,
275                 "number.omni must pass linter: {:?}",
276                 report.errors
277             );
278             assert_eq!(
279                 report.blocks_found.len(),
280                 4,
281                 "Must find all 4 blocks in number.omni"
282             );
283             let audit = report
284                 .audit_score
285                 .as_ref()
286                 .expect("Audit score must be computed");
287             assert!(
288                 audit.eligible_to_compile,
289                 "number.omni must be eligible: {:?}",
290                 audit.gate
291             );
292             assert_eq!(audit.gate, QuadStateGate::Proceed);
293             assert_eq!(
294                 audit.composite_score, 100,
295                 "number.omni must achieve composite score 100"
296             );
297         }
298     }
299
300     #[test]
301     fn test_linter_number_tape_process_ribbon_passes() {
302         let logger = DiagnosticLogger::new(false);
303         let linter = OmniLinter::new(logger);
304         let path = crate::core::filetype::resolve_track_path(
305             "lexicon/01_nouns/ideas/types/number.tape",
306         );
307         if path.exists() {
308             let report = linter.lint_file(&path).expect("Linting number.tape");
```

```
            assert!(
                report.is_valid,
                "number.tape must pass linter: {:?}",
                report.errors
            );
            assert!(
                !report.blocks_found.is_empty(),
                "Must detect head in number.tape"
            );
            let audit = report
                .audit_score
                .as_ref()
                .expect("Audit score must be computed");
            assert!(
                audit.eligible_to_compile,
                "number.tape must be eligible: {:?}",
                audit.gate
            );
            assert_eq!(audit.gate, QuadStateGate::Proceed);
            assert_eq!(
                audit.composite_score, 100,
                "number.tape must achieve composite score 100"
            );
        }
    }

    #[test]
    fn test_linter_conjunctions_suite_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger);
        let files = [
            "lexicon/07_conjunctions/caveat_branches.omni",
            "lexicon/07_conjunctions/caveat_branches.tape",
            "lexicon/07_conjunctions/inseparable_pairs.omni",
            "lexicon/07_conjunctions/inseparable_pairs.tape",
            "lexicon/07_conjunctions/pipeline_junctions.omni",
            "lexicon/07_conjunctions/pipeline_junctions.tape",
            "lexicon/07_conjunctions/alternative_branches.omni",
            "lexicon/07_conjunctions/alternative_branches.tape",
            "lexicon/07_conjunctions/relational_attributions.omni",
            "lexicon/07_conjunctions/relational_attributions.tape",
        ];

        for f in &files {
            let path = crate::core::filetype::resolve_track_path(f);
            if path.exists() {
                let report = linter.lint_file(&path).unwrap();
                assert!(
                    report.is_valid,
                    "{} must pass linter: {:?}",
                    f, report.errors
                );
                assert!(
                    report.errors.is_empty(),
                    "{} must have 0 syntax errors: {:?}",
                    f, report.errors
                );
            }
        }
    }

    #[test]
    fn test_linter_spatial_80_column_strict_rejection() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger).with_strict_geometry(true);
        let wide_code = format!("# {}\n", "A".repeat(85));
        let report = linter.lint_source("wide.omni", &wide_code);
        assert!(!report.is_valid);
        assert!(report.errors.iter().any(|e| e.contains("Line exceeds 80 columns")));
    }

    #[test]
    fn test_linter_assembly_preflight_partition_bounds() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger);
        let bad_partition_tape = r#"
            = [Tape]: < Lexicon > & [ Test ] {
                WHERE {partition} = [ 17 ] ::
```

```
                WHO {System} :: WHAT {Test} :: WHEN {Event} :: WHY (Test) :: HOW [+1:PROCEED]
            }
            [ ROLL 1 ] : [ Action ] : [ Output ]
            [ ROLL 2 ] : [ Commit Status ] : [ +1:PROCEED ]
        "#;
        let report = linter.lint_source("bad_partition.tape", bad_partition_tape);
        assert!(report.warnings.iter().any(|w| w.contains("below minimum 64 cells") || w.contains("not a power of 2")));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/detector.rs

RS | 241 LINES | 919 WORDS | 8.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Block Boundary Detector & Semantic Slice Extractor )
// ============================================================================
//! Universal Block Boundary Detector
//!
//! Locates and slices the four canonical blocks across both `.omni` Source Reels
//! and `.tape` Process Ribbons using dual-layer detection:
//! 1. Comment Header `#` Delimiters (`# BLOCK 1`, `# BLOCK 2`, `# ================`).
//! 2. Semantic Code Keywords (`Opening`, `Body`, `Closing`, `= [Tape]:`, `[ ROLL 1 ]`).
//!
//! # Foundational Invariant:
//! Any code appearing prior to the `Opening` scope (or before the first operational
//! roll in `.tape`) is deterministically categorized as Block 1 (Metadata).

#[derive(Debug, Clone)]
pub struct BlockSlice {
    pub start_line: usize,
    pub end_line: usize,
    pub content: String,
}

#[derive(Debug, Clone)]
pub struct FileBlockSlices {
    pub metadata: BlockSlice,
    pub opening: BlockSlice,
    pub body: BlockSlice,
    pub closing: BlockSlice,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( BlockDetector Definition )
// ============================================================================

pub struct BlockDetector;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Extraction Logic for Reels and Tapes )
// ============================================================================

impl BlockDetector {
    /// Detects and extracts all four block slices from source code.
    pub fn detect_blocks(content: &str, is_tape: bool) -> FileBlockSlices {
        if is_tape {
            Self::detect_tape_blocks(content)
        } else {
            Self::detect_omni_blocks(content)
        }
    }

    /// Detects the 4-block lifecycle in `.omni` Source Reels.
    fn detect_omni_blocks(content: &str) -> FileBlockSlices {
        let lines: Vec<&str> = content.lines().collect();
        let total_lines = lines.len();

        let mut opening_line = None;
        let mut body_line = None;
        let mut closing_line = None;

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            let upper = trimmed.to_uppercase();

            // Detect Opening Block boundary
            if opening_line.is_none() {
                if upper.contains("BLOCK 2")
                    || upper.contains("OPENING BLOCK")
                    || trimmed.starts_with("Opening {")
                    || trimmed.starts_with("Opening\n")
                    || trimmed == "Opening"
```

```
                {
                    opening_line = Some(line_num);
                    continue;
                }
            }

            // Detect Body Block boundary
            if opening_line.is_some() && body_line.is_none() {
                if upper.contains("BLOCK 3")
                    || upper.contains("BODY BLOCK")
                    || trimmed.starts_with("Body [")
                    || trimmed.starts_with("Body {")
                    || trimmed == "Body"
                {
                    body_line = Some(line_num);
                    continue;
                }
            }

            // Detect Closing Block boundary
            if body_line.is_some() && closing_line.is_none() {
                if upper.contains("BLOCK 4")
                    || upper.contains("CLOSING BLOCK")
                    || trimmed.starts_with("Closing {")
                    || trimmed.starts_with("Closing\n")
                    || trimmed.starts_with("Closing (")
                    || trimmed == "Closing"
                {
                    closing_line = Some(line_num);
                }
            }
        }

        let l_open = opening_line.unwrap_or(total_lines + 1);
        let l_body = body_line.unwrap_or(total_lines + 1);
        let l_close = closing_line.unwrap_or(total_lines + 1);

        // Invariant: Everything prior to Opening is Metadata (Block 1)
        let meta_end = if l_open > 1 && l_open <= total_lines {
            // If opening line is a comment header like '# BLOCK 2', backtrack slightly
            l_open - 1
        } else {
            total_lines
        };

        let meta_slice = Self::slice_lines(&lines, 1, meta_end);
        let open_slice = if l_open <= total_lines {
            let end = if l_body > l_open && l_body <= total_lines { l_body - 1 } else { total_lines };
            Self::slice_lines(&lines, l_open, end)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let body_slice = if l_body <= total_lines {
            let end = if l_close > l_body && l_close <= total_lines { l_close - 1 } else { total_lines };
            Self::slice_lines(&lines, l_body, end)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let close_slice = if l_close <= total_lines {
            Self::slice_lines(&lines, l_close, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        FileBlockSlices {
            metadata: meta_slice,
            opening: open_slice,
            body: body_slice,
            closing: close_slice,
        }
    }

    /// Detects the Tape Head vs. Tape Rolls lifecycle in `.tape` Process Ribbons.
    fn detect_tape_blocks(content: &str) -> FileBlockSlices {
        let lines: Vec<&str> = content.lines().collect();
        let total_lines = lines.len();
```

```
        let mut rolls_indices: Vec<(usize, String)> = Vec::new();
        let mut head_end_line = None;

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            let upper = trimmed.to_uppercase();

            if upper.starts_with("[ ROLL") || upper.starts_with("ROLL_") {
                if head_end_line.is_none() {
                    head_end_line = Some(if line_num > 1 { line_num - 1 } else { 1 });
                }
                rolls_indices.push((line_num, trimmed.to_string()));
            }
        }

        let head_end = head_end_line.unwrap_or(total_lines);
        let meta_slice = Self::slice_lines(&lines, 1, head_end);

        if rolls_indices.is_empty() {
            return FileBlockSlices {
                metadata: meta_slice,
                opening: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
                body: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
                closing: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
            };
        }

        // Roll 1 serves as the initial setup / opening condition
        let (first_roll_line, _) = rolls_indices[0];
        let (last_roll_line, ref last_roll_text) = rolls_indices[rolls_indices.len() - 1];

        let is_last_commit = last_roll_text.to_uppercase().contains("COMMIT STATUS")
            || last_roll_text.to_uppercase().contains("COMMIT_STATUS");

        let open_end = if rolls_indices.len() > 1 {
            rolls_indices[1].0 - 1
        } else if is_last_commit {
            first_roll_line
        } else {
            total_lines
        };

        let open_slice = Self::slice_lines(&lines, first_roll_line, open_end);

        let body_slice = if rolls_indices.len() > 2 {
            let body_start = rolls_indices[1].0;
            let body_end = if is_last_commit { last_roll_line - 1 } else { total_lines };
            Self::slice_lines(&lines, body_start, body_end)
        } else if rolls_indices.len() == 2 && !is_last_commit {
            Self::slice_lines(&lines, rolls_indices[1].0, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let close_slice = if is_last_commit {
            Self::slice_lines(&lines, last_roll_line, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        FileBlockSlices {
            metadata: meta_slice,
            opening: open_slice,
            body: body_slice,
            closing: close_slice,
        }
    }

    fn slice_lines(lines: &[&str], start: usize, end: usize) -> BlockSlice {
        if start == 0 || end == 0 || start > end || start > lines.len() {
            return BlockSlice { start_line: 0, end_line: 0, content: String::new() };
        }
        let clamped_end = end.min(lines.len());
        let slice = lines[(start - 1)..clamped_end].join("\n");
        BlockSlice {
            start_line: start,
```

```
            end_line: clamped_end,
            content: slice,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// BlockDetector sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/mod.rs

RS | 61 LINES | 209 WORDS | 2.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structural Architecture Subsystem Coordinator )
// ============================================================================
//! Structural Architecture Linter Subsystem
//!
//! Enforces full lifecycle architectures:
//! - [`omni`]: 4-Block Lifecycle (Metadata -> Opening -> Body -> Closing) for `.omni` reels.
//! - [`tape`]: Sequential Process Ribbon Lifecycle (Tape Head -> Rolls 1..N -> Commit Status) for `.tape`.

pub mod detector;
pub mod omni;
pub mod tape;

pub use detector::{BlockDetector, BlockSlice, FileBlockSlices};
pub use omni::OmniReelLinter;
pub use tape::TapeLinter;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Block Hierarchy Declarations )
// ============================================================================

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum BlockKind {
    Metadata = 1,
    Opening = 2,
    Body = 3,
    Closing = 4,
}

impl BlockKind {
    pub fn name(&self) -> &'static str {
        match self {
            BlockKind::Metadata => "BLOCK 1: METADATA",
            BlockKind::Opening => "BLOCK 2: OPENING",
            BlockKind::Body => "BLOCK 3: BODY",
            BlockKind::Closing => "BLOCK 4: CLOSING",
        }
    }
}

#[derive(Debug, Clone)]
pub struct DetectedBlock {
    pub kind: BlockKind,
    pub line_number: usize,
    pub header_raw: String,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Coordination Invariants )
// ============================================================================
// Structural submodules coordinate full-file lifecycle verification.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// Structural architecture sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/omni.rs

RS | 175 LINES | 680 WORDS | 7.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode 4-Block Source Reel (.omni) Structural Linter )
// ============================================================================
//! OmniCode 4-Block Source Reel Structural Linter
//!
//! Validates canonical 4-Block architecture for `.omni` files:
//! - BLOCK 1: METADATA BLOCK (Identity, Version, Invariant Axioms, Asides)
//! - BLOCK 2: OPENING BLOCK  (Context, Preconditions, State Initialization)
//! - BLOCK 3: BODY BLOCK     (Core Transformations, Causality Vectors, Logic)
//! - BLOCK 4: CLOSING BLOCK  (Verification, Postconditions, Teardown, Commit)

use crate::diagnostics::linter::{BlockKind, DetectedBlock, LintReport};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniReelLinter Definition )
// ============================================================================

pub struct OmniReelLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal 4-Block Sequence & Semantic Scope Verification )
// ============================================================================

impl OmniReelLinter {
    /// Lints a `.omni` source reel string against universal 4-Block invariants.
    pub fn lint(file_path: &str, content: &str, report: &mut LintReport) {
        let mut blocks = Vec::new();

        for (idx, line) in content.lines().enumerate() {
            let line_num = idx + 1;
            // Ignore indented lines (e.g. sub-reels or inner scopes)
            if line.starts_with(' ') || line.starts_with('\t') {
                continue;
            }
            let upper = line.to_uppercase();

            if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Metadata,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Opening,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 3") || upper.contains("BODY BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Body,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 4") || upper.contains("CLOSING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Closing,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
        }

        // Check 1: Must contain all 4 blocks
        let has_metadata = blocks.iter().any(|b| b.kind == BlockKind::Metadata);
        let has_opening = blocks.iter().any(|b| b.kind == BlockKind::Opening);
        let has_body = blocks.iter().any(|b| b.kind == BlockKind::Body);
        let has_closing = blocks.iter().any(|b| b.kind == BlockKind::Closing);

        if !has_metadata {
```

```
            report.errors.push("Missing BLOCK 1: METADATA BLOCK".into());
        }
        if !has_opening {
            report.errors.push("Missing BLOCK 2: OPENING BLOCK".into());
        }
        if !has_body {
            report.errors.push("Missing BLOCK 3: BODY BLOCK".into());
        }
        if !has_closing {
            report.errors.push("Missing BLOCK 4: CLOSING BLOCK".into());
        }

        // Check 2: Verify sequential ordering
        for i in 1..blocks.len() {
            if blocks[i].kind <= blocks[i - 1].kind {
                let err = format!(
                    "Block order violation at line {}: {} appeared after {}",
                    blocks[i].line_number,
                    blocks[i].kind.name(),
                    blocks[i - 1].kind.name()
                );
                report.errors.push(err);
            }
        }

        // Check 3: Semantic Content Analysis Across Detected Block Regions
        if has_metadata && has_opening && has_body && has_closing && report.errors.is_empty() {
            let meta_start = blocks.iter().find(|b| b.kind == BlockKind::Metadata).unwrap().line_number;
            let open_start = blocks.iter().find(|b| b.kind == BlockKind::Opening).unwrap().line_number;
            let body_start = blocks.iter().find(|b| b.kind == BlockKind::Body).unwrap().line_number;
            let close_start = blocks.iter().find(|b| b.kind == BlockKind::Closing).unwrap().line_number;

            let lines: Vec<&str> = content.lines().collect();

            let get_slice = |start: usize, end: usize| -> String {
                if start <= end && end <= lines.len() {
                    lines[(start - 1)..(end - 1)].join("\n")
                } else if start <= lines.len() {
                    lines[(start - 1)..].join("\n")
                } else {
                    String::new()
                }
            };

            let meta_slice = get_slice(meta_start, open_start);
            let open_slice = get_slice(open_start, body_start);
            let body_slice = get_slice(body_start, close_start);
            let close_slice = get_slice(close_start, lines.len() + 1);

            // Block 1 Validation
            let has_module_decl = meta_slice.contains("= [Module]:")
                || meta_slice.contains("Module <")
                || meta_slice.contains("Module [")
                || meta_slice.contains("Scroll <");
            if !has_module_decl {
                report.warnings.push(
                    "BLOCK 1: Missing formal module identity declaration (e.g. '= [Module]: < ... > & [ ... ]').".into()
                );
            }
            if !meta_slice.contains("Version") && !meta_slice.contains("version") {
                report.warnings.push("BLOCK 1: Missing version specification in metadata context.".into());
            }
            if file_path.contains("lexicon") || file_path.contains("manifest") {
                if !meta_slice.contains("Exodus") && !meta_slice.contains("Scripture") && !meta_slice.contains("Witness")
{
                    report.warnings.push(
                        "BLOCK 1: Lexicon/Covenant specification should ground its invariant witness (e.g. '< Scripture {
... } >').".into()
                    );
                }
            }

            // Block 2 Validation
            if !open_slice.contains("Opening") {
                report.errors.push("BLOCK 2: Missing 'Opening' block scope declaration.".into());
            }
            if !open_slice.contains("preconditions") && !open_slice.contains("Preconditions") &&
!open_slice.contains("+1") {
```

```
                report.warnings.push("BLOCK 2: Opening block should explicitly verify preconditions / armed state.".into());
            }

            // Block 3 Validation
            if !body_slice.contains("Body") {
                report.errors.push("BLOCK 3: Missing 'Body' block scope declaration.".into());
            }

            // Block 4 Validation
            if !close_slice.contains("Closing") {
                report.errors.push("BLOCK 4: Missing 'Closing' block scope declaration.".into());
            }
            if !close_slice.contains("commit_status") && !close_slice.contains("Commit Status") && !close_slice.contains("Status =") {
                report.warnings.push("BLOCK 4: Closing block should affirm terminal commit status (e.g. '[ Commit Status ]').".into());
            }
        }

        report.blocks_found = blocks;
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Sealing )
// ================================================================================
// OmniReelLinter sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/tape.rs

RS | 230 LINES | 849 WORDS | 10.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Process Tape (.tape) Structural & Ribbon Linter )
// ============================================================================
//! OmniCode Process Tape (.tape) Structural Linter
//!
//! Validates canonical Turing-style process ribbons for `.tape` files:
//! 1. Leading Tape Head (Covenant & Process Manifest):
//!    - Identity declaration (`= [Tape]:` or `[Tape.Name]`).
//!    - Six Cardinal Clauses: `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`.
//!    - Invariant Scripture Witness (`< Scripture { ... } >`).
//! 2. Subordinate Operational Rolls (`[ ROLL 1 ]` .. `[ ROLL N ]` or `roll_n`):
//!    - Strictly ascending numerical order.
//!    - Non-empty operational predicates with SVO verbs or transduction.
//!    - Living Aside attached to every roll.
//! 3. Terminal Commit Status Roll:
//!    - Final roll must seal the tape in an affirmative Fourfold Commit Status.

use crate::diagnostics::linter::{BlockKind, DetectedBlock, LintReport};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( TapeLinter Definition )
// ============================================================================

pub struct TapeLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Process Ribbon Lifecycle & Monotonic Roll Progression )
// ============================================================================

impl TapeLinter {
    /// Lints a `.tape` process ribbon string against canonical tape invariants.
    pub fn lint(file_path: &str, content: &str, report: &mut LintReport) {
        let lines: Vec<&str> = content.lines().collect();

        // ------------------------------------------------------------------
        // 1. Detect and Validate Tape Head
        // ------------------------------------------------------------------
        let mut tape_head_line = None;
        let mut tape_head_raw = String::new();
        let mut head_text_accumulator = String::new();
        let mut in_head = false;
        let mut is_block_manifest = false;

        let mut roll_headers: Vec<(usize, usize, String)> = Vec::new(); // (line_num, roll_num, text)

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();

            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            if tape_head_line.is_none() {
                if trimmed.starts_with("= [Tape")
                    || trimmed.starts_with("[Tape.")
                    || trimmed.starts_with("Tape [")
                    || trimmed.starts_with("Tape <")
                {
                    tape_head_line = Some(line_num);
                    tape_head_raw = trimmed.to_string();
                    in_head = true;
                    is_block_manifest = trimmed.ends_with('{');
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                    continue;
                }
            }

            if in_head {
```

```
                if is_block_manifest {
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                    if trimmed == "}" || trimmed.ends_with('}') {
                        in_head = false;
                    }
                } else if trimmed.starts_with("WHERE")
                    || trimmed.starts_with("WHEN")
                    || trimmed.starts_with("WHY")
                    || trimmed.starts_with("HOW")
                    || trimmed.starts_with("WHO")
                    || trimmed.starts_with("WHAT")
                    || trimmed.starts_with("< Scripture")
                    || trimmed.starts_with("::")
                {
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                } else {
                    in_head = false;
                }
            }

            // Detect Subordinate Rolls (e.g. `[ ROLL 1 ]` or `roll_1`)
            let upper = trimmed.to_uppercase();
            if upper.starts_with("[ ROLL") || upper.starts_with("ROLL_") {
                let roll_num = Self::extract_roll_number(&upper);
                roll_headers.push((line_num, roll_num, trimmed.to_string()));
            }
        }

        // ---------------------------------------------------------------------
        // Validate Tape Head Manifest
        // ---------------------------------------------------------------------
        if let Some(h_line) = tape_head_line {
            report.blocks_found.push(DetectedBlock {
                kind: BlockKind::Metadata,
                line_number: h_line,
                header_raw: tape_head_raw,
            });

            // Check Cardinal Clauses
            let head_upper = head_text_accumulator.to_uppercase();
            let missing_clauses: Vec<&str> = ["WHO", "WHAT", "WHEN", "WHERE", "WHY", "HOW"]
                .iter()
                .filter(|&&c| !head_upper.contains(c))
                .copied()
                .collect();

            if !missing_clauses.is_empty() {
                report.warnings.push(format!(
                    "TAPE HEAD (line {h_line}): Missing cardinal clause(s): {}. \
                     OmniCode Process Tapes must specify complete cardinal coordinates.",
                    missing_clauses.join(", ")
                ));
            }

            // Check Scripture / Covenant Witness
            if file_path.contains("lexicon") || file_path.contains("covenant") {
                if !head_text_accumulator.contains("Scripture")
                    && !head_text_accumulator.contains("Exodus")
                    && !head_text_accumulator.contains("covenant")
                {
                    report.warnings.push(
                        format!("TAPE HEAD (line {h_line}): Lexicon tape should anchor its invariant witness (e.g. '<
Scripture {{ ... }} >').")
                    );
                }
            }
        } else {
            report.errors.push("Missing Tape Head manifest declaration (e.g. '= [Tape]: ...' or '[Tape.Name]').".into());
        }

        // ---------------------------------------------------------------------
        // Validate Subordinate Rolls
        // ---------------------------------------------------------------------
        if roll_headers.is_empty() {
            report.errors.push("Missing operational subordinate rolls in Process Tape (expected '[ ROLL 1 ]'
...).".into());
```

```
        } else {
            report.blocks_found.push(DetectedBlock {
                kind: BlockKind::Body,
                line_number: roll_headers[0].0,
                header_raw: format!("ROLLS: {} subordinate rolls declared", roll_headers.len()),
            });

            // Verify strict sequential ordering of rolls
            for i in 1..roll_headers.len() {
                let (prev_line, prev_num, _) = roll_headers[i - 1];
                let (curr_line, curr_num, _) = roll_headers[i];
                if curr_num <= prev_num && curr_num > 0 {
                    report.errors.push(format!(
                        "Roll sequence violation at line {curr_line}: Roll {curr_num} appeared after Roll {prev_num} at line {prev_line}."
                    ));
                }
            }

            // ---------------------------------------------------------------
            // Validate Terminal Commit Status Roll
            // ---------------------------------------------------------------
            let (last_line, last_num, ref last_text) = roll_headers[roll_headers.len() - 1];
            let last_upper = last_text.to_uppercase();
            let is_commit_roll = last_upper.contains("COMMIT STATUS") || last_upper.contains("COMMIT_STATUS");

            if !is_commit_roll {
                report.warnings.push(format!(
                    "ROLL {last_num} (line {last_line}): Terminal roll should be an explicit Commit Status roll (e.g. '[ ROLL {last_num} ] : [ Commit Status ] : [ ... ]')."
                ));
            } else {
                report.blocks_found.push(DetectedBlock {
                    kind: BlockKind::Closing,
                    line_number: last_line,
                    header_raw: last_text.clone(),
                });

                // Check Fourfold state machine if branching
                if content.contains("Commit Status") && content.contains('|') {
                    let commit_slice = &content[content.find("Commit Status").unwrap()..];
                    let commit_upper = commit_slice.to_uppercase();
                    let mut missing_states = Vec::new();
                    if !commit_upper.contains("PROCEED") { missing_states.push("PROCEED (+1)"); }
                    if !commit_upper.contains("AWAIT") { missing_states.push("AWAIT (0)"); }
                    if !commit_upper.contains("HALT") { missing_states.push("HALT (NULL)"); }
                    if !commit_upper.contains("ROLLBACK") { missing_states.push("ROLLBACK (-1)"); }

                    if !missing_states.is_empty() {
                        report.errors.push(format!(
                            "COMMIT STATUS (line {last_line}): Incomplete fourfold state machine. When using conditional branching, all four states must be defined. Missing: {}.",
                            missing_states.join(", ")
                        ));
                    }
                }
            }
        }
    }

    fn extract_roll_number(text: &str) -> usize {
        let parts: Vec<&str> = text.split_whitespace().collect();
        for (i, p) in parts.iter().enumerate() {
            if *p == "ROLL" && i + 1 < parts.len() {
                if let Ok(num) = parts[i + 1].trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                    return num;
                }
            }
            if p.starts_with("ROLL_") {
                if let Ok(num) = p[5..].trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                    return num;
                }
            }
        }
        0
    }
}
```

```
// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Sealing )
// ================================================================================
// TapeLinter sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/mod.rs RS | 84 LINES | 265 WORDS | 3.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 2 Opening & Setup Subsystem Coordinator )
// ============================================================================
//! OmniCode Block 2 Opening Subsystem
//!
//! Evaluates precondition equilibrium and execution container setup:
//! - [`reel`]: Source Reel (`.omni`) Opening block verification.
//! - [`tape_setup`]: Process Tape (`.tape`) Roll 1 initial setup verification.
//! - [`score`]: `OpeningScore` model on continuous $[-100 \dots +100]$ space.

pub mod reel;
pub mod score;
pub mod tape_setup;

pub use reel::ReelOpeningLinter;
pub use score::OpeningScore;
pub use tape_setup::TapeSetupLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

pub struct OpeningLinter;

impl OpeningLinter {
    /// Evaluates and scores Block 2 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeSetupLinter::evaluate(file_path, &blocks.opening.content, config)
        } else {
            ReelOpeningLinter::evaluate(file_path, &blocks.opening.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block2_opening_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = OpeningLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 2 Opening must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }

    #[test]
    fn test_character_tape_block2_setup_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = OpeningLinter::evaluate("character.tape", &content, true, &config);
```

```
            assert_eq!(
                score.total, 100,
                "character.tape Roll 1 setup must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/reel.rs

RS | 106 LINES | 508 WORDS | 4.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 2 Opening Linter & Precondition Verifier )
// ================================================================================
//! Source Reel (.omni) Block 2 Opening Linter
//!
//! Validates execution environment initialization:
//! - Input payload imposition (`{ Glyph ]: ...`)
//! - Invariant entity allocation bound to cell container (`[ Character ]: < Character > & [ Cell ]`)
//! - Concrete memory cell bounds (`[ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte }`)
//! - Hardware transducer bridging (`{ 8 Bits <=> 5 Trits }`)
//! - Target destination mapping (`::@ 0x01` / `::@ 0x000B8000`)
//! - Precondition equilibrium proofs (`[ (+1) plus (-1) <=> 0 ]`)

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::OpeningScore;

pub struct ReelOpeningLinter;

impl ReelOpeningLinter {
    /// Evaluates and scores Block 2 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let mut entity = 0i32;
        let mut cell = 0i32;
        let mut substrate = 0i32;
        let mut preconditions = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 2: Opening scope entirely (-50)".into());
            return OpeningScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Entity & Input Payload Allocation (max 25 pts)
        let has_entity = slice.contains("Entity") || slice.contains("Character") || slice.contains("Glyph") ||
slice.contains("Input");
        if has_entity {
            entity += 25;
            breakdown.push("✓ Typed entity & input payload allocated (+25/25)".into());
        } else {
            entity -= 15;
            breakdown.push("✗ Missing typed entity or input allocation (-15/25)".into());
        }

        // 2. Cell Container Sizing (max 20 pts)
        let has_cell = slice.contains("Cell") || slice.contains("Memory") || slice.contains("Partition");
        if has_cell {
            cell += 20;
            breakdown.push("✓ Concrete memory cell container allocated (+20/20)".into());
        } else {
            cell -= 10;
            breakdown.push("✗ Missing memory cell container definition (-10/20)".into());
        }

        // 3. Substrate Bridge / Transducer (max 20 pts)
        let has_substrate = slice.contains("Transducer") || slice.contains("<=>") || slice.contains("Trits") ||
slice.contains("Bits");
        if has_substrate {
            substrate += 20;
            breakdown.push("✓ Substrate transducer / bridge armed (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit substrate transducer bridge (0/20)".into());
        }

        // 4. Preconditions & Target Mapping (max 20 pts)
        let has_preconditions = slice.contains("Preconditions") || slice.contains("preconditions") ||
slice.contains("+1") || slice.contains("Equilibrium");
        let has_target = slice.contains("Target") || slice.contains("::@") || slice.contains("Stdout") ||
```

```
slice.contains("VGA");
        if has_preconditions && has_target {
            preconditions += 20;
            breakdown.push("✓ Precondition equilibrium verified & target hardware mapped (+20/20)".into());
        } else if has_preconditions || has_target {
            preconditions += 15;
            breakdown.push("✓ Preconditions or target destination declared (+15/20)".into());
        } else {
            breakdown.push("✗ Missing explicit precondition check (-10/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 2 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Block 2 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Living Asides attached to opening stanzas (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on opening definitions (-5/5)".into());
        }

        OpeningScore::new(entity, cell, substrate, preconditions, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/score.rs

RS | 71 LINES | 267 WORDS | 2.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 2 Opening Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 2 Opening Score Model
//!
//! Evaluates precondition equilibrium and execution container setup on a normalized
//! $[-100 \dots +100]$ continuous scalar metric space:
//! 1. Entity & Input Payload Allocation (max 25 pts)
//! 2. Cell Container Sizing & Capacity Bounds (max 20 pts)
//! 3. Substrate Bridge / Transducer Arming (max 20 pts)
//! 4. Hardware Targets & Precondition Equilibrium Proofs (max 20 pts)
//! 5. Spatial Geometry & Living Aside Intent Oracles (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct OpeningScore {
    pub entity_allocation_score: i32,
    pub cell_container_score: i32,
    pub substrate_bridge_score: i32,
    pub preconditions_target_score: i32,
    pub spatial_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl OpeningScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Equilibrium Certified)",
            75..=89 => "A (Compliant Setup)",
            50..=74 => "B (Adequate Setup)",
            20..=49 => "C (Unverified Preconditions)",
            0..=19 => "D (Deficient Containers)",
            _ => "F (Active Boundary Hazard / Debt)",
        }
    }

    pub fn new(
        entity: i32,
        cell: i32,
        substrate: i32,
        preconditions: i32,
        spatial: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (entity + cell + substrate + preconditions + spatial).clamp(-100, 100);
        Self {
            entity_allocation_score: entity,
            cell_container_score: cell,
            substrate_bridge_score: substrate,
            preconditions_target_score: preconditions,
            spatial_asides_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// OpeningScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/tape_setup.rs

RS | 112 LINES | 456 WORDS | 4.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Initial Roll / Precondition Setup Linter )
// ============================================================================
//! Process Tape (.tape) Initial Roll Setup Linter
//!
//! Evaluates the initial allocation and precondition setup of a `.tape` ribbon:
//! - Roll 1 declaration (`[ ROLL 1 ] : [ Inscribe ] : ...`)
//! - Capturing external input payload into cell container
//! - Substrate parity and single-cell bound preconditions
//! - Attached Living Aside cognitive oracle

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::OpeningScore;

pub struct TapeSetupLinter;

impl TapeSetupLinter {
    /// Evaluates Roll 1 setup for `.tape` files.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let mut entity = 0i32;
        let mut cell = 0i32;
        let mut substrate = 0i32;
        let mut preconditions = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Roll 1 initial allocation (-50)".into());
            return OpeningScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Roll 1 Declaration & Entity Allocation (max 25 pts)
        let upper = slice.to_uppercase();
        let has_roll1 = upper.contains("ROLL 1") || upper.contains("ROLL_1");
        if has_roll1 {
            entity += 25;
            breakdown.push("✓ Roll 1 initial state allocation declared (+25/25)".into());
        } else {
            entity -= 15;
            breakdown.push("✗ Missing Roll 1 initial setup declaration (-15/25)".into());
        }

        // 2. Cell / Word Container Specification (max 20 pts)
        let has_cell = slice.contains("Cell")
            || slice.contains("cell")
            || slice.contains("Word")
            || slice.contains("word")
            || slice.contains("Character`")
            || slice.contains("Number`")
            || slice.contains("Entity`");
        if has_cell {
            cell += 20;
            breakdown.push("✓ Memory container target specified (+20/20)".into());
        } else {
            cell -= 10;
            breakdown.push("✗ Missing container specification (-10/20)".into());
        }

        // 3. Operational Inscription / Transduction (max 20 pts)
        let has_op = slice.contains("INSCRIBE") || slice.contains("-->") || slice.contains("Inscribe") ||
slice.contains("Alloc");
        if has_op {
            substrate += 20;
            breakdown.push("✓ Initial inscription / input capture vector wired (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit inscription vector in setup (0/20)".into());
        }

```

```
        // 4. Precondition / State Guard Verification (max 20 pts)
        let has_guard = slice.contains("Glyph")
            || slice.contains("Quantity")
            || slice.contains("Input")
            || slice.contains("Cell")
            || slice.contains("Word")
            || slice.contains("Status");
        if has_guard {
            preconditions += 20;
            breakdown.push("✓ Initial condition inputs & container parity verified (+20/20)".into());
        } else {
            breakdown.push("⚠ Implicit initial preconditions (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Roll 1 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Roll 1 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Machine-readable Living Aside attached to Roll 1 (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Aside on Roll 1 (-5/5)".into());
        }

        OpeningScore::new(entity, cell, substrate, preconditions, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/mod.rs RS 84 LINES 264 WORDS 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 3 Body & Tape Rolls Subsystem Coordinator )
// ============================================================================
//! OmniCode Block 3 Body Subsystem
//!
//! Evaluates operational transformations, causal pipelines, and tape rolls:
//! - [`reel`]: Source Reel (`.omni`) Body block verification.
//! - [`tape_rolls`]: Process Tape (`.tape`) operational rolls verification.
//! - [`score`]: `BodyScore` model on continuous $[-100 \dots +100]$ space.

pub mod reel;
pub mod score;
pub mod tape_rolls;

pub use reel::ReelBodyLinter;
pub use score::BodyScore;
pub use tape_rolls::TapeRollsLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

pub struct BodyLinter;

impl BodyLinter {
    /// Evaluates and scores Block 3 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeRollsLinter::evaluate(file_path, &blocks.body.content, config)
        } else {
            ReelBodyLinter::evaluate(file_path, &blocks.body.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block3_body_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = BodyLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 3 Body must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }

    #[test]
    fn test_character_tape_block3_rolls_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = BodyLinter::evaluate("character.tape", &content, true, &config);
            assert_eq!(
```

```
                score.total, 100,
                "character.tape Rolls section must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/reel.rs RS 115 LINES 508 WORDS 4.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 3 Body Linter & Causal Vector Verifier )
// ============================================================================
//! Source Reel (.omni) Block 3 Body Linter
//!
//! Validates operational transformations, causal pipelines, and state evolution:
//! - Active all-caps SVO operational verbs (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`)
//! - Directional vectors (`-->`, `<=>`, `^--`, `v--`)
//! - State-prime progression notation (`Character``, `Cell``, `Target``)
//! - Adversative guard caveats (`! GUARD --> [ Status` = +1:PROCEED ] | ...`)
//! - Attached Living Aside intent oracles on operational stanzas

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::BodyScore;

pub struct ReelBodyLinter;

impl ReelBodyLinter {
    /// Evaluates and scores Block 3 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let mut svo = 0i32;
        let mut vectors = 0i32;
        let mut primes = 0i32;
        let mut guards = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 3: Body scope entirely (-50)".into());
            return BodyScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. SVO Operational Verbs (max 25 pts)
        let has_inscribe = slice.contains("INSCRIBE");
        let has_transduce = slice.contains("TRANSDUCE");
        let has_project = slice.contains("PROJECT");
        let has_sequence = slice.contains("SEQUENCE");
        let active_verbs = [has_inscribe, has_transduce, has_project, has_sequence].iter().filter(|&&v| v).count();

        if active_verbs >= 3 || slice.contains("Body [") || slice.contains("ROLL") {
            svo += 25;
            breakdown.push("✓ Active SVO operational verbs wired (+25/25)".into());
        } else if active_verbs > 0 {
            svo += 15;
            breakdown.push("✓ Partial SVO verbs present (+15/25)".into());
        } else {
            svo -= 10;
            breakdown.push("✗ Missing canonical SVO operational verbs (-10/25)".into());
        }

        // 2. Directional Vectors & Transition Syntax (max 20 pts)
        let has_forward = slice.contains("-->");
        let has_bidirectional = slice.contains("<=>");
        let has_logic = slice.contains('&') || slice.contains('|');

        if has_forward && (has_bidirectional || has_logic) {
            vectors += 20;
            breakdown.push("✓ Directional causal vectors ('-->', '<=>') verified (+20/20)".into());
        } else if has_forward {
            vectors += 15;
            breakdown.push("✓ Forward causal vector wired (+15/20)".into());
        } else {
            vectors -= 10;
            breakdown.push("✗ Missing directional causal vectors (-10/20)".into());
        }

        // 3. State-Prime Progression (max 20 pts)
        let has_primes = slice.contains('``');
```

```
        if has_primes {
            primes += 20;
            breakdown.push("✓ State-prime immutable succession affirmed (+20/20)".into());
        } else {
            breakdown.push("⚠ No state-prime notation detected in body transitions (0/20)".into());
        }

        // 4. Adversative Guard Caveats & Equilibrium Testing (max 20 pts)
        let has_guard = slice.contains("GUARD") || slice.contains('!') || slice.contains("Compare");
        if has_guard {
            guards += 20;
            breakdown.push("✓ Adversative guard caveat '! GUARD' armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Body transitions operate without explicit guard caveats (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 3 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Block 3 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Living Asides attached to operational rolls (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on operational rolls (-5/5)".into());
        }

        BodyScore::new(svo, vectors, primes, guards, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/score.rs

RS | 71 LINES | 266 WORDS | 2.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 3 Body Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 3 Body & Tape Rolls Score Model
//!
//! Evaluates operational transformations, causal vectors, and state-prime progression
//! on a normalized $[-100 \dots +100]$ continuous scalar metric space:
//! 1. SVO Operational Verbs (max 25 pts)
//! 2. Directional Vectors & Boundary Transitions (max 20 pts)
//! 3. State-Prime Progression & Immutability (max 20 pts)
//! 4. Adversative Guard Caveats & Equilibrium Testing (max 20 pts)
//! 5. Spatial Geometry & Roll-Level Living Asides (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct BodyScore {
    pub svo_verbs_score: i32,
    pub directional_vectors_score: i32,
    pub state_prime_score: i32,
    pub guard_caveats_score: i32,
    pub spatial_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl BodyScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Causally Complete)",
            75..=89 => "A (Compliant Body)",
            50..=74 => "B (Adequate Logic)",
            20..=49 => "C (Unguarded Transitions)",
            0..=19 => "D (Weak Operations)",
            _ => "F (Active Operational Hazard / Debt)",
        }
    }

    pub fn new(
        svo: i32,
        vectors: i32,
        primes: i32,
        guards: i32,
        spatial: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (svo + vectors + primes + guards + spatial).clamp(-100, 100);
        Self {
            svo_verbs_score: svo,
            directional_vectors_score: vectors,
            state_prime_score: primes,
            guard_caveats_score: guards,
            spatial_asides_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// BodyScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/tape_rolls.rs

RS | 116 LINES | 512 WORDS | 4.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Declared Rolls Section Linter )
// ============================================================================
//! Process Tape (.tape) Declared Rolls Section Linter
//!
//! Validates the operational body rolls of a `.tape` file, strictly distinguishing
//! the Tape Head manifest from the sequential operational rolls:
//! - Ensures Tape Head is sealed before rolls begin
//! - Monotonically increasing roll numbering (`[ ROLL 1 ]` -> `[ ROLL 2 ]` ...)
//! - SVO operational verbs in active positions
//! - Attached Living Aside intent oracle `( ... )` on every operational roll
//! - Guard caveats and directional vectors

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::BodyScore;

pub struct TapeRollsLinter;

impl TapeRollsLinter {
    /// Evaluates the operational rolls section of a `.tape` file.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let mut svo = 0i32;
        let mut vectors = 0i32;
        let mut primes = 0i32;
        let mut guards = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing operational rolls section (-50)".into());
            return BodyScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. SVO Operational Verbs across Rolls (max 25 pts)
        let upper = slice.to_uppercase();
        let has_verbs = upper.contains("INSCRIBE")
            || upper.contains("TRANSDUCE")
            || upper.contains("PROJECT")
            || upper.contains("SEQUENCE");

        if has_verbs {
            svo += 25;
            breakdown.push("✓ Operational SVO verbs wired into tape rolls (+25/25)".into());
        } else {
            svo -= 10;
            breakdown.push("✗ Missing SVO operational verbs in tape rolls (-10/25)".into());
        }

        // 2. Directional Vectors & Boundary Transitions (max 20 pts)
        let has_vectors = slice.contains("-->") || slice.contains("<=>");
        if has_vectors {
            vectors += 20;
            breakdown.push("✓ Causal transition vectors ('-->', '<=>') present (+20/20)".into());
        } else {
            vectors -= 10;
            breakdown.push("✗ Missing causal vectors in operational rolls (-10/20)".into());
        }

        // 3. State-Prime Progression (max 20 pts)
        let has_primes = slice.contains('`');
        if has_primes {
            primes += 20;
            breakdown.push("✓ State-prime succession affirmed across rolls (+20/20)".into());
        } else {
            breakdown.push("⚠ No state-prime symbols detected in operational rolls (0/20)".into());
        }

        // 4. Guard Caveats & Verification Rolls (max 20 pts)
```

```
        let has_guard = slice.contains("GUARD") || slice.contains('!') || slice.contains("Compare");
        if has_guard {
            guards += 20;
            breakdown.push("✓ Adversative guard caveat '! GUARD' armed in rolls (+20/20)".into());
        } else {
            breakdown.push("⚠ Rolls execute without explicit guard caveats (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides on Every Roll (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All operational roll lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Operational roll lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let rolls_count = lines.iter().filter(|l| {
            let u = l.trim().to_uppercase();
            u.starts_with("[ ROLL") || u.starts_with("ROLL_")
        }).count();

        let asides_count = lines.iter().filter(|l| l.contains('(') && l.contains(')')).count();

        if rolls_count > 0 && asides_count >= rolls_count {
            spatial += 5;
            breakdown.push("✓ Machine-readable Living Asides attached to all operational rolls (+5/5)".into());
        } else if asides_count > 0 {
            spatial += 3;
            breakdown.push("⚠ Living Asides present on some rolls (+3/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on operational rolls (-5/5)".into());
        }

        BodyScore::new(svo, vectors, primes, guards, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/assembly.rs

RS | 133 LINES | 560 WORDS | 6.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Assembly Pre-flight Invariant Linter & Silicon Certification )
// ============================================================================
//! OmniCode Assembly Pipeline Pre-flight Linter
//!
//! Serves as an invariant verification aid before physical lowering, memory allocation,
//! and machine word synthesis in [`OmniAssembler`]:
//! 1. Memory Partition Pre-flight:
//!    - Verifies `WHERE {partition} = [ N ]` specifies a power of 2 >= 64 cells.
//! 2. Silicon MMIO Address Pre-flight:
//!    - Validates hardware locators (`::@ 0x...`) match known ports (e.g. 0x01 Stdout, 0x000B8000 VGA).
//! 3. Hardware CPU Register Pre-flight:
//!    - Validates that physical registers (`%self`, `%caller`, `%acc`, `%swarm`, `%disc`, etc.)
//!      correspond to the 13 canonical registers in the ISA.
//! 4. Delimiter Stack Pre-flight:
//!    - Early boundary leak detection before tokenization to avert assembler panics.

use crate::diagnostics::linter::LintReport;

pub struct AssemblyLinter;

impl AssemblyLinter {
    /// Pre-flights source text before assembly lowering, asserting hardware invariants.
    pub fn preflight(content: &str, report: &mut LintReport) {
        // ------------------------------------------------------------------------
        // 1. Partition Size Verification
        // ------------------------------------------------------------------------
        if let Some(where_idx) = content.find("WHERE") {
            let where_slice = &content[where_idx..];
            if where_slice.to_lowercase().contains("partition") {
                for token in where_slice.split_whitespace().take(10) {
                    if let Ok(num) = token.trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                        if num > 0 {
                            if num < 64 {
                                report.warnings.push(format!(
                                    "Assembly Pre-flight: Partition size [{num}] is below minimum 64 cells. Clamping to 64."
                                ));
                            } else if !num.is_power_of_two() {
                                report.warnings.push(format!(
                                    "Assembly Pre-flight: Partition size [{num}] is not a power of 2. May cause memory alignment penalties."
                                ));
                            }
                            break;
                        }
                    }
                }
            }
        }

        // ------------------------------------------------------------------------
        // 2. Silicon MMIO Address Validation
        // ------------------------------------------------------------------------
        for (idx, line) in content.lines().enumerate() {
            let line_num = idx + 1;
            if line.contains("::@") {
                if let Some(pos) = line.find("::@") {
                    let locator_slice = &line[pos + 3..];
                    let addr_token = locator_slice.split_whitespace().next().unwrap_or("");
                    let addr_clean = addr_token.trim_matches(|c: char| c == '}' || c == ']' || c == ',' || c == ';');

                    if addr_clean.starts_with("0x") || addr_clean.starts_with("0X") {
                        if let Ok(addr) = usize::from_str_radix(&addr_clean[2..], 16) {
                            match addr {
                                0x00 => {} // Stdin port
                                0x01 => {} // Stdout port
                                0x02 => {} // Stderr port
                                0x000B8000 => {} // VGA text buffer
                                0x00020000 => {} // Virtual framebuffer
                                0x03F8 => {} // COM1 serial port
                                other => {
```

```
                                    report.warnings.push(format!(
                                        "Assembly Pre-flight (line {line_num}): Unmapped silicon MMIO target 0x{other:X}.
\
                                         Verify custom hardware mapping before bare-metal deployment."
                                    ));
                                }
                            }
                        }
                    }
                }
            }

            // ---------------------------------------------------------------------
            // 3. Delimiter Stack Parity Pre-flight (via Lexer & BoundaryType)
            // ---------------------------------------------------------------------
            let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
            let mut lexer = crate::parser::lexer::Lexer::new(content, logger);
            if let Ok(tokens) = lexer.tokenize() {
                let mut stack: Vec<(char, usize, usize)> = Vec::new();
                for tok in tokens {
                    let (ch, is_open) = match tok.token_type {
                        crate::parser::lexer::TokenType::LBracket => ('[', true),
                        crate::parser::lexer::TokenType::RBracket => (']', false),
                        crate::parser::lexer::TokenType::LBrace => ('{', true),
                        crate::parser::lexer::TokenType::RBrace => ('}', false),
                        crate::parser::lexer::TokenType::LAngle => ('<', true),
                        crate::parser::lexer::TokenType::RAngle => ('>', false),
                        _ => continue,
                    };

                    if is_open {
                        stack.push((ch, tok.line, tok.col));
                    } else if let Some((open_ch, _open_line, _open_col)) = stack.pop() {
                        if crate::core::boundaries::BoundaryType::from_delimiters(open_ch, ch).is_err() {
                            report.errors.push(format!(
                                "Assembly Pre-flight (line {}): Illegal boundary crossing '{open_ch} ... {ch}'.",
                                tok.line
                            ));
                        }
                    } else {
                        report.errors.push(format!(
                            "Assembly Pre-flight (line {}): Unmatched closing delimiter '{ch}'.",
                            tok.line
                        ));
                    }
                }

                for (unclosed, line, col) in stack {
                    report.errors.push(format!(
                        "Assembly Pre-flight: Unclosed delimiter '{unclosed}' opened at L{line}:C{col} reaches EOF."
                    ));
                }
            }
        }
    }

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ================================================================================
// AssemblyLinter sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/mod.rs

RS | 19 LINES | 63 WORDS | 0.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Assembly Pre-flight Subsystem Coordinator )
// ============================================================================
//! Assembly Pre-flight Linter Subsystem
//!
//! Enforces hardware invariants, memory partition sizing, MMIO mappings,
//! and delimiter stack integrity before machine lowering.

pub mod assembly;

pub use assembly::AssemblyLinter;

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// Preflight subsystem sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/mod.rs

RS | 84 LINES | 272 WORDS | 3.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 4 Closing & Commit Status Subsystem Coordinator )
// ============================================================================
//! OmniCode Block 4 Closing Subsystem
//!
//! Evaluates postcondition verification proofs and commit status completeness:
//! - [`reel`]: Source Reel (`.omni`) Closing block verification.
//! - [`tape_commit`]: Process Tape (`.tape`) terminal commit status roll verification.
//! - [`score`]: `ClosingScore` model on continuous $[-100 \dots +100]$ space.

pub mod reel;
pub mod score;
pub mod tape_commit;

pub use reel::ReelClosingLinter;
pub use score::ClosingScore;
pub use tape_commit::TapeCommitLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

pub struct ClosingLinter;

impl ClosingLinter {
    /// Evaluates and scores Block 4 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeCommitLinter::evaluate(file_path, &blocks.closing.content, config)
        } else {
            ReelClosingLinter::evaluate(file_path, &blocks.closing.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block4_closing_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = ClosingLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 4 Closing must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }

    #[test]
    fn test_character_tape_block4_commit_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = ClosingLinter::evaluate("character.tape", &content, true, &config);
```

```
            assert_eq!(
                score.total, 100,
                "character.tape Roll 6 Commit Status must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/reel.rs

RS | 113 LINES | 551 WORDS | 5.1 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 4 Closing Linter & Postcondition Verifier )
// ================================================================================
//! Source Reel (.omni) Block 4 Closing Linter
//!
//! Validates postcondition proofs, circuit breakers, and commit state machines:
//! - Operational roll verification proofs (`[ Verify ]: [ [ ROLL 1 ] = PROCEED :: ... ]`)
//! - Invariant validation & capacity bounds (`[ Validate ]: { Character` <= [ 1 Cell ] }`)
//! - Autonomous circuit breaker tripwiring (`[ Tripwire ]: ... --> [ ARMED ] | ... --> [ TRIPPED ]`)
//! - Diagnostic debug snapshot (`[ Debug ]: { ... }`)
//! - Episodic journal inscription (`[ Log ]: [ Journal ]: INSCRIBE { ... }`)
//! - Fourfold state machine (`+1:PROCEED`, `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`)

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::ClosingScore;

pub struct ReelClosingLinter;

impl ReelClosingLinter {
    /// Evaluates and scores Block 4 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let mut verify = 0i32;
        let mut validate = 0i32;
        let mut tripwire = 0i32;
        let mut fourfold = 0i32;
        let mut telemetry = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 4: Closing scope entirely (-50)".into());
            return ClosingScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Operational Verification Proofs (max 20 pts)
        let has_verify = slice.contains("Verify") || slice.contains("verify") || slice.contains("PROCEED");
        if has_verify {
            verify += 20;
            breakdown.push("✓ Operational verification proofs present (+20/20)".into());
        } else {
            verify -= 10;
            breakdown.push("✗ Missing operational verification proof stanza (-10/20)".into());
        }

        // 2. Invariant Validation & Bounds Parity (max 20 pts)
        let has_validate = slice.contains("Validate") || slice.contains("validate") || slice.contains("Enclosed") ||
slice.contains("Cell");
        if has_validate {
            validate += 20;
            breakdown.push("✓ Invariant bounds & substrate parity validated (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit validation bounds check (0/20)".into());
        }

        // 3. Autonomous Circuit Breaker / Tripwiring (max 20 pts)
        let has_tripwire = slice.contains("Tripwire") || slice.contains("ARMED") || slice.contains("TRIPPED") ||
slice.contains("circuit_breaker");
        if has_tripwire {
            tripwire += 20;
            breakdown.push("✓ Zero-tolerance tripwire circuit breaker armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Execution operates without autonomous circuit breaker (0/20)".into());
        }

        // 4. Fourfold State Machine Completeness (max 25 pts)
        let upper = slice.to_uppercase();
        let states = ["PROCEED", "AWAIT", "HALT", "ROLLBACK"];
        let present_states = states.iter().filter(|&&s| upper.contains(s)).count();

```

```
        if present_states == 4 {
            fourfold += 25;
            breakdown.push("✓ Complete Balanced Ternary Fourfold State Machine (T4) affirmed (+25/25)".into());
        } else if present_states >= 2 {
            let pts = (present_states as i32) * 5;
            fourfold += pts;
            breakdown.push(format!("⚠ Partial commit state machine ({present_states}/4 states) (+{pts}/25)"));
        } else if upper.contains("COMMIT") || upper.contains("STATUS") {
            fourfold += 10;
            breakdown.push("✓ Basic commit status declared (+10/25)".into());
        } else {
            fourfold -= 15;
            breakdown.push("✗ Missing terminal commit status definition (-15/25)".into());
        }

        // 5. Telemetry & Journal Logging + Living Asides (max 15 pts)
        let has_telemetry = (slice.contains("Debug") || slice.contains("telemetry"))
            && (slice.contains("Log") || slice.contains("Journal"));
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            telemetry += 8;
            breakdown.push(format!("✓ All Block 4 lines <= {max_cols} columns (+8/8)"));
        } else {
            telemetry -= 8;
            breakdown.push(format!("✗ Block 4 lines exceed {max_cols}-column spatial ruler (-8/8)"));
        }

        if has_telemetry {
            telemetry += 7;
            breakdown.push("✓ Diagnostic debug snapshot & episodic logging verified (+7/7)".into());
        } else {
            telemetry += 4;
            breakdown.push("✓ Standard sealing gate verified (+4/7)".into());
        }

        ClosingScore::new(verify, validate, tripwire, fourfold, telemetry, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/score.rs

RS | 71 LINES | 271 WORDS | 2.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 4 Closing Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 4 Closing & Commit Status Score Model
//!
//! Evaluates postcondition verification proofs, tripwire circuit breakers,
//! and fourfold commit status completeness on a normalized $[-100 \dots +100]$ space:
//! 1. Operational Verification Proofs (max 20 pts)
//! 2. Invariant Validation & Substrate Parity (max 20 pts)
//! 3. Zero-Tolerance Tripwire Circuit Breaker (max 20 pts)
//! 4. Fourfold Commit State Machine Completeness (max 25 pts)
//! 5. Debug Telemetry, Journal Logging & Living Asides (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ClosingScore {
    pub verification_proofs_score: i32,
    pub invariant_validation_score: i32,
    pub tripwire_breaker_score: i32,
    pub fourfold_state_machine_score: i32,
    pub telemetry_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl ClosingScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Covenant Sealed & Verified)",
            75..=89 => "A (Compliant Closing)",
            50..=74 => "B (Adequate Verification)",
            20..=49 => "C (Unverified Commits)",
            0..=19 => "D (Missing Circuit Breakers)",
            _ => "F (Active Closing Hazard / Unsealed State)",
        }
    }

    pub fn new(
        verify: i32,
        validate: i32,
        tripwire: i32,
        fourfold: i32,
        telemetry: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (verify + validate + tripwire + fourfold + telemetry).clamp(-100, 100);
        Self {
            verification_proofs_score: verify,
            invariant_validation_score: validate,
            tripwire_breaker_score: tripwire,
            fourfold_state_machine_score: fourfold,
            telemetry_asides_score: telemetry,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// ClosingScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/tape_commit.rs

RS | 112 LINES | 499 WORDS | 4.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Terminal Commit Status Roll Linter )
// ============================================================================
//! Process Tape (.tape) Terminal Commit Status Roll Linter
//!
//! Validates the terminal sealing roll of a `.tape` process ribbon:
//! - Terminal roll identity (`[ ROLL N ] : [ Commit Status ] : [ ... ]`)
//! - Verification of preceding rolls achieving `+1:PROCEED`
//! - Tripwire circuit breaker arming
//! - Fourfold state machine completeness:
//!   `+1:PROCEED`, `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`
//! - Attached Living Aside intent oracle

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::ClosingScore;

pub struct TapeCommitLinter;

impl TapeCommitLinter {
    /// Evaluates the terminal commit status roll of a `.tape` file.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let mut verify = 0i32;
        let mut validate = 0i32;
        let mut tripwire = 0i32;
        let mut fourfold = 0i32;
        let mut telemetry = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing terminal commit status roll (-50)".into());
            return ClosingScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Terminal Roll Identity & Verification (max 20 pts)
        let upper = slice.to_uppercase();
        let is_commit = upper.contains("COMMIT STATUS") || upper.contains("COMMIT_STATUS");
        if is_commit {
            verify += 20;
            breakdown.push("✓ Terminal Commit Status roll verified (+20/20)".into());
        } else {
            verify -= 10;
            breakdown.push("✗ Terminal roll is not a designated Commit Status roll (-10/20)".into());
        }

        // 2. Preceding Roll Affirmation (max 20 pts)
        let has_affirm = slice.contains("Verify") || slice.contains("PROCEED") || slice.contains("+1");
        if has_affirm {
            validate += 20;
            breakdown.push("✓ Affirmative execution condition verified (+20/20)".into());
        } else {
            breakdown.push("⚠ Implicit execution conditions (0/20)".into());
        }

        // 3. Tripwire Circuit Breaker Guard (max 20 pts)
        let has_tripwire = slice.contains("Tripwire") || slice.contains("ARMED") || slice.contains("TRIPPED");
        if has_tripwire {
            tripwire += 20;
            breakdown.push("✓ Zero-tolerance tripwire circuit breaker armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Terminal commit operates without explicit tripwire check (0/20)".into());
        }

        // 4. Fourfold State Machine Completeness (max 25 pts)
        let states = ["PROCEED", "AWAIT", "HALT", "ROLLBACK"];
        let present_states = states.iter().filter(|&&s| upper.contains(s)).count();

        if present_states == 4 {
            fourfold += 25;
```

```
            breakdown.push("✓ Complete Balanced Ternary Fourfold State Machine (T4) affirmed (+25/25)".into());
        } else if present_states >= 2 {
            let pts = (present_states as i32) * 5;
            fourfold += pts;
            breakdown.push(format!("⚠ Partial commit state machine ({present_states}/4 states) (+{pts}/25)"));
        } else if is_commit {
            fourfold += 10;
            breakdown.push("✓ Basic commit status declared (+10/25)".into());
        } else {
            fourfold -= 15;
            breakdown.push("✗ Missing fourfold commit state machine (-15/25)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            telemetry += 8;
            breakdown.push(format!("✓ All commit status lines <= {max_cols} columns (+8/8)"));
        } else {
            telemetry -= 8;
            breakdown.push(format!("✗ Commit status lines exceed {max_cols}-column spatial ruler (-8/8)"));
        }

        let has_aside = slice.contains('(') && slice.contains(')');
        if has_aside {
            telemetry += 7;
            breakdown.push("✓ Machine-readable Living Aside attached to terminal commit (+7/7)".into());
        } else {
            telemetry -= 5;
            breakdown.push("✗ Missing Living Aside on terminal commit (-5/7)".into());
        }

        ClosingScore::new(verify, validate, tripwire, fourfold, telemetry, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/grammar.rs

RS | 142 LINES | 589 WORDS | 6.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Lexical Syntax & Syntactic Grammar Linter )
// ================================================================================
//! OmniCode Lexical Syntax & Grammar Linter
//!
//! Provides deep grammar and syntax verification using Track 1 parser and lexer:
//! 1. Full Lexical Tokenization Validation:
//!    - Detects unclosed strings, unclosed asides `( ... )`, illegal characters, and boundary leaks.
//! 2. Syntactic Grammar AST Validation:
//!    - Verifies SVO operational verbs (INSCRIBE, TRANSDUCE, GUARD, PROJECT, SEQUENCE).
//!    - Verifies directional vectors (`-->`, `<=>`, `::`, `&`, `|`).
//!    - Verifies state-prime synchronization (`Name``).
//!    - Validates hardware locators (`::@ 0x...`).
//!    - Validates delimiter balance across all 6 ranks of boundaries.

use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::diagnostics::linter::LintReport;
use crate::parser::lexer::{Lexer, TokenType};
use crate::parser::parser::Parser;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( GrammarLinter Definition )
// ================================================================================

pub struct GrammarLinter;

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Token Stream Hygiene & AST Grammar Verification )
// ================================================================================

impl GrammarLinter {
    /// Performs comprehensive lexical syntax and syntactic grammar analysis on source code.
    pub fn lint(
        file_path: &str,
        content: &str,
        logger: DiagnosticLogger,
        debugger: OmniDebugger,
        report: &mut LintReport,
    ) {
        // --------------------------------------------------------------------
        // 1. Lexical Tokenization Pass
        // --------------------------------------------------------------------
        let mut lexer = Lexer::new(content, logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(toks) => toks,
            Err(e) => {
                report.errors.push(format!("Syntax Grammar Error (Lexical): {e}"));
                return;
            }
        };

        // --------------------------------------------------------------------
        // 2. Token Stream Grammatical Pattern Checks
        // --------------------------------------------------------------------
        let mut in_roll = false;
        let mut current_roll_line = 0;
        let mut roll_has_aside = false;

        for (idx, tok) in tokens.iter().enumerate() {
            // Check State Prime Grammatical Hygiene
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id.ends_with('`') && id.len() > 1 {
                    // Valid state-prime notation
                }
            }

            // Track Rolls and verify each has an attached Living Aside
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id == "ROLL" || id == "Roll" {
```

```
                    let is_roll_decl = idx + 2 < tokens.len() && matches!(tokens[idx + 2].token_type,
TokenType::InRelationTo);
                    if is_roll_decl {
                        if in_roll && !roll_has_aside && current_roll_line > 0 {
                            report.warnings.push(format!(
                                "Grammar Contract (line {current_roll_line}): Operational roll missing attached Living
Aside '( ... )' oracle."
                            ));
                        }
                        in_roll = true;
                        current_roll_line = tok.line;
                        roll_has_aside = false;
                    }
                }
            }

            if in_roll {
                if let TokenType::Aside(_) = tok.token_type {
                    roll_has_aside = true;
                }
            }

            // Check SVO Verb Grammar: Verbs must be followed by target containers
            // Operational verbs in active positions are ALL-CAPS: INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE
            if let TokenType::Identifier(ref id) = tok.token_type {
                if matches!(id.as_str(), "INSCRIBE" | "TRANSDUCE" | "PROJECT" | "SEQUENCE") {
                    let is_chamber = idx > 0 && matches!(tokens[idx - 1].token_type, TokenType::LBracket);
                    if !is_chamber && idx + 1 < tokens.len() {
                        let next = &tokens[idx + 1];
                        if !next.token_type.is_open_boundary() && !matches!(&next.token_type, TokenType::Identifier(_)) {
                            report.warnings.push(format!(
                                "Grammar Error (line {}): SVO Verb '{}' must be followed by a target container or
operand, found '{:?}'.",
                                tok.line, id, next.token_type
                            ));
                        }
                    }
                }
            }
        }

        // Final roll aside check
        if in_roll && !roll_has_aside && current_roll_line > 0 {
            report.warnings.push(format!(
                "Grammar Contract (line {current_roll_line}): Operational roll missing attached Living Aside '( ... )'
oracle."
            ));
        }

        // ---------------------------------------------------------------------
        // 3. Syntactic Parser AST Pass (for non-empty code)
        // ---------------------------------------------------------------------
        // For .omni files, parse the entire program AST
        if file_path.ends_with(".omni") || content.contains("= [Module]:") || content.contains("Module <") {
            let mut parser = Parser::new(tokens, debugger.clone());
            match parser.parse_program() {
                Ok(_ast) => {
                    // AST successfully parsed and validated
                }
                Err(e) => {
                    report.errors.push(format!("Syntactic Grammar Error: {e}"));
                }
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// GrammarLinter sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/mod.rs

RS | 34 LINES | 144 WORDS | 1.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Syntax, Spatial & Grammar Subsystem Coordinator )
// ============================================================================
//! Syntax, Spatial Geometry & Grammar Linter Subsystem
//!
//! Enforces lexical syntax, grammatical AST correctness, and spatial geometry:
//! - [`spatial`]: 80-column IDE dual side-panel ruler and bracket identifier naming hygiene.
//! - [`grammar`]: Lexer token stream hygiene, AST parser verification, SVO verbs, and Living Asides.

pub mod grammar;
pub mod spatial;

pub use grammar::GrammarLinter;
pub use spatial::{SpatialLintConfig, SpatialLinter};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Coordination Invariants )
// ============================================================================
// Syntax submodules coordinate pre-compilation syntactic and layout hygiene.

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Pipeline )
// ============================================================================
// Coordinated across passes 1 and 3 of the OmniLinter pipeline.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// Syntax subsystem sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/spatial.rs

RS | 109 LINES | 443 WORDS | 4.8 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Spatial Geometry, Ruler Invariants & Syntax Hygiene Linter )
// ================================================================================
//! OmniCode Spatial Geometry & Layout Linter
//!
//! Enforces physical spatial invariants across all OmniCode files:
//! 1. 80-Column Spatial Ruler: Guarantees legibility in IDE viewports with dual side panels.
//! 2. Delimiter & Bracket Hygiene: Disallows snake_case inside brackets (natural spaced phrases).
//! 3. First-Class Intent Asides: Mandates Living Asides `( ... )` for AI cognitive prompts and test oracles.
//! 4. Multiline Compound Sentence Formatting: Flags run-on clauses that break columnar rhythm.

use crate::diagnostics::linter::LintReport;

/// Configuration for spatial geometry and layout linting.
#[derive(Debug, Clone)]
pub struct SpatialLintConfig {
    /// Maximum allowed column width (default: 80).
    pub max_columns: usize,
    /// Whether column limit violations constitute fatal errors or advisory warnings.
    pub strict_column_limit: bool,
}

impl Default for SpatialLintConfig {
    fn default() -> Self {
        Self {
            max_columns: 80,
            strict_column_limit: false,
        }
    }
}

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( SpatialLinter Definition & Constructor )
// ================================================================================

pub struct SpatialLinter;

impl SpatialLinter {
    /// Lints source text against physical spatial geometry and syntax hygiene rules.
    pub fn lint(content: &str, report: &mut LintReport, config: &SpatialLintConfig) {
        let lines: Vec<&str> = content.lines().collect();

        // ------------------------------------------------------------------------
        // 1. 80-Column Spatial Ruler Verification
        // ------------------------------------------------------------------------
        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let col_count = line.chars().count();
            if col_count > config.max_columns {
                let msg = format!(
                    "Spatial Geometry (line {line_num}): Line exceeds {max} columns ({col_count} cols). \
                     OmniCode mandates <= {max} columns for dual side panel IDE viewports.",
                    max = config.max_columns
                );
                if config.strict_column_limit {
                    report.errors.push(msg);
                } else {
                    report.warnings.push(msg);
                }
            }
        }

        // ------------------------------------------------------------------------
        // 2. First-Class Intent Asides Presence (Canon 1)
        // ------------------------------------------------------------------------
        let total_lines = lines.len();
        let has_any_asides = content.contains('(') && content.contains(')');
        if total_lines > 15 && !has_any_asides {
            report.warnings.push(
                "Canon 1: Missing first-class intent Asides '(...)'. OmniCode mandates Asides \
                 for human-AI pronunciation, system prompt seeds, and pre-test oracles.".into(),
```

```
            );
        }

        // ---------------------------------------------------------------------
        // 3. Snake_case Hygiene Inside Brackets
        // ---------------------------------------------------------------------
        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                continue;
            }
            if let Some(start_bracket) = line.find('[') {
                if let Some(end_bracket) = line[start_bracket..].find(']') {
                    let bracket_content = &line[start_bracket + 1..start_bracket + end_bracket];
                    for word in bracket_content.split_whitespace() {
                        if word.contains('_') && !word.starts_with("0x") && !word.starts_with("roll_") {
                            report.warnings.push(format!(
                                "Syntax Hygiene (line {line_num}): Identifier '{word}' uses snake_case with underscores
inside brackets. \
                                 In OmniCode, use natural spaced phrases (e.g. '[ ... ]') without underscores."
                            ));
                            break;
                        }
                    }
                }
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// SpatialLinter sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/head.rs

RS | 247 LINES | 953 WORDS | 10.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Block 1 Tape Head & Cardinal Coordinates Linter )
// ============================================================================
//! Process Tape (.tape) Block 1 Tape Head Linter
//!
//! Evaluates and scores the Tape Head manifest for `.tape` Process Ribbons.
//! While `.omni` files represent declarative Source Reels, `.tape` files represent
//! sequential execution ribbons. Both embody the exact same ontological covenant
//! and foundation, but with process ribbon mechanics:
//! 1. Formal Tape Head Declaration (`= [Tape]: < ... > & [ ... ] {`).
//! 2. Defined Operational Space & Partitions (`WHERE { ... Cell } = [ 1 ]`).
//! 3. Covenant Witness Grounding (`< Scripture { ... } >` anchor).
//! 4. The Six Cardinal Coordinates (`WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`)
//!    with operational verbs (`--> INSCRIBE`) and causal directionals.
//! 5. Spatial Geometry (<= 80 columns) and bracket naming hygiene.

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::MetadataScore;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( TapeHeadLinter Definition & Slice Extractor )
// ============================================================================

pub struct TapeHeadLinter;

impl TapeHeadLinter {
    /// Extracts the text slice corresponding exclusively to the Tape Head.
    pub fn extract_slice(content: &str) -> (usize, usize, String) {
        let lines: Vec<&str> = content.lines().collect();
        if lines.is_empty() {
            return (0, 0, String::new());
        }

        let mut start_idx = None;
        let mut end_idx = None;

        for (idx, line) in lines.iter().enumerate() {
            let trimmed = line.trim();
            if start_idx.is_none() {
                if trimmed.starts_with("= [Tape")
                    || trimmed.starts_with("[Tape.")
                    || trimmed.starts_with("Tape [")
                    || trimmed.starts_with("Tape <")
                {
                    start_idx = Some(idx);
                }
            } else {
                let upper = trimmed.to_uppercase();
                if upper.starts_with("[ ROLL")
                    || upper.starts_with("ROLL_")
                    || trimmed.starts_with("# DECLARED ROLLS")
                {
                    end_idx = Some(idx);
                    break;
                }
            }
        }

        let start = start_idx.unwrap_or(0);
        let end = end_idx.unwrap_or(lines.len());
        (start + 1, end, lines[start..end].join("\n"))
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Evaluation Engine for Process Tape Head Invariants )
// ============================================================================

impl TapeHeadLinter {
    /// Evaluates and scores Block 1 for a `.tape` Process Ribbon.
```

```rust
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> MetadataScore {
        let mut identity = 0u32;
        let mut provenance = 0u32;
        let mut witness = 0u32;
        let mut ontology = 0u32;
        let mut spatial = 0u32;
        let mut breakdown = Vec::new();

        // ------------------------------------------------------------------
        // 1. Identity & Formal Tape Declaration (max 20 pts)
        // ------------------------------------------------------------------
        let has_tape_decl = slice.contains("= [Tape]:")
            || slice.contains("[Tape.")
            || slice.contains("Tape [")
            || slice.contains("Tape <");
        if has_tape_decl {
            identity += 15;
            breakdown.push("✓ Formal Process Tape Head declared (+15/15)".into());
        } else {
            breakdown.push("✗ Missing formal tape head declaration (0/15)".into());
        }

        let has_tape_context = (slice.contains('<') && slice.contains('>'))
            && (slice.contains('[') && slice.contains(']'));
        if has_tape_context {
            identity += 5;
            breakdown.push("✓ Dual-domain tape contextualization affirmed (+5/5)".into());
        } else {
            breakdown.push("✗ Missing dual domain context in tape declaration (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 2. Operational Environment & Topology (max 25 pts)
        // ------------------------------------------------------------------
        let has_defined_space = slice.contains("WHERE")
            && (slice.contains("Cell") || slice.contains("Partition") || slice.contains('['));
        if has_defined_space {
            provenance += 10;
            breakdown.push("✓ Defined space / cell partition bounds declared (+10/10)".into());
        } else {
            breakdown.push("✗ Missing defined space or cell bounds in tape head (0/10)".into());
        }

        let has_domain_context = slice.contains("Lexicon") || slice.contains("Kernel") || slice.contains("Domain");
        if has_domain_context {
            provenance += 10;
            breakdown.push("✓ Execution domain context declared (+10/10)".into());
        } else {
            breakdown.push("✗ Missing execution domain context (0/10)".into());
        }

        let has_comments = slice.contains("TOPOLOGY")
            || slice.contains("PROCESS TAPE")
            || slice.lines().any(|l| l.trim().starts_with('#'));
        if has_comments {
            provenance += 5;
            breakdown.push("✓ Architectural topology commentary inscribed (+5/5)".into());
        } else {
            breakdown.push("✗ Missing architectural topology header commentary (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 3. Covenant & Witness Invariant Grounding (max 20 pts)
        // ------------------------------------------------------------------
        let has_scripture = slice.contains("< Scripture") || slice.contains("Exodus");
        if has_scripture {
            witness += 15;
            breakdown.push("✓ Invariant Scripture / Covenant witness anchored (+15/15)".into());
        } else {
            breakdown.push("✗ Missing scripture/covenant witness in tape head (0/15)".into());
        }

        let has_witness_aside = slice.contains("Invariant Witness")
            || (has_scripture && slice.contains('(') && slice.contains(')'));
```

```
        if has_witness_aside {
            witness += 5;
            breakdown.push("✓ Machine-readable Witness Aside oracle attached (+5/5)".into());
        } else {
            breakdown.push("✗ Missing witness Aside oracle (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 4. Six Cardinal Coordinates & Operational Verbs (max 20 pts)
        // ------------------------------------------------------------------
        let upper = slice.to_uppercase();
        let cardinal_clauses = ["WHO", "WHAT", "WHEN", "WHERE", "WHY", "HOW"];
        let present_clauses = cardinal_clauses.iter().filter(|&&c| upper.contains(c)).count();

        if present_clauses == 6 {
            ontology += 15;
            breakdown.push(
                "✓ All Six Cardinal Coordinates (WHO, WHAT, WHEN, WHERE, WHY, HOW) present (+15/15)".into(),
            );
        } else {
            let pts = (present_clauses as u32) * 2;
            ontology += pts;
            breakdown.push(format!(
                "⚠ Cardinal coordinates incomplete ({present_clauses}/6) (+{pts}/15)"
            ));
        }

        let has_op_verbs = upper.contains("INSCRIBE") || upper.contains("PROCEED") || upper.contains("-->");
        if has_op_verbs {
            ontology += 5;
            breakdown.push("✓ Operational SVO verb and directional vectors wired (+5/5)".into());
        } else {
            breakdown.push("✗ Missing operational verbs or directional vectors in cardinal clauses (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 5. Spatial Geometry & Syntax Hygiene (max 15 pts)
        // ------------------------------------------------------------------
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Tape Head lines <= {max_cols} columns (+10/10)"));
        } else {
            breakdown.push(format!("✗ Tape Head lines exceed {max_cols}-column spatial ruler (0/10)"));
        }

        let snake_violation = lines.iter().any(|line| {
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                return false;
            }
            if let Some(start) = line.find('[') {
                if let Some(end) = line[start..].find(']') {
                    let inside = &line[start + 1..start + end];
                    for w in inside.split_whitespace() {
                        if w.contains('_') && !w.starts_with("0x") && !w.starts_with("roll_") {
                            return true;
                        }
                    }
                }
            }
            false
        });

        if !snake_violation {
            spatial += 5;
            breakdown.push("✓ Natural spaced identifier hygiene observed (+5/5)".into());
        } else {
            breakdown.push("✗ snake_case identifiers found inside brackets (0/5)".into());
        }

        MetadataScore::new(identity, provenance, witness, ontology, spatial, breakdown)
    }
}
```

```
// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Tape Head Block 1 Linting )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_tape_head_extraction_and_empty_check() {
        let (start, end, slice) = TapeHeadLinter::extract_slice("");
        assert_eq!(start, 0);
        assert_eq!(end, 0);
        assert!(slice.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/mod.rs

RS | 112 LINES | 388 WORDS | 4.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 1 Metadata & Tape Head Invariant Subsystem Coordinator )
// ============================================================================
//! OmniCode Block 1 Metadata & Tape Head Subsystem
//!
//! Organizes Block 1 verification into dedicated, domain-specific modules:
//! - [`score`]: `MetadataScore` invariant model, point totals, and rating tiers.
//! - [`reel`]: Source Reel (`.omni`) Module Manifest & Invariant Type inspection.
//! - [`head`]: Process Tape (`.tape`) Tape Head & Cardinal Coordinates inspection.
//!
//! While `.omni` and `.tape` have different syntaxes and structures, both
//! embody the same proper ontological understanding: foundational identity,
//! provenance, covenant witness grounding, invariants/clauses, and spatial hygiene.

pub mod head;
pub mod reel;
pub mod score;

pub use head::TapeHeadLinter;
pub use reel::ReelMetadataLinter;
pub use score::MetadataScore;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::LintReport;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( MetadataLinter Coordinator Definition )
// ============================================================================

pub struct MetadataLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Polymorphic Dispatch Across Source Reels and Process Tapes )
// ============================================================================

impl MetadataLinter {
    /// Evaluates and scores Block 1 for either a Source Reel or Process Tape,
    /// selecting the dedicated linter while preserving unified understanding.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
        _report: &mut LintReport,
    ) -> MetadataScore {
        let blocks = crate::diagnostics::linter::structure::detector::BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeHeadLinter::evaluate(file_path, &blocks.metadata.content, config)
        } else {
            ReelMetadataLinter::evaluate(file_path, &blocks.metadata.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {

    #[test]
    fn test_character_omni_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint character.omni");
            assert!(report.is_valid, "character.omni must be valid");
```

```
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "character.omni Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }

    #[test]
    fn test_character_tape_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint character.tape");
            assert!(report.is_valid, "character.tape must be valid");
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "character.tape Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/reel.rs

RS | 250 LINES | 995 WORDS | 10.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 1 Metadata & Invariant Type Linter )
// ============================================================================
//! Source Reel (.omni) Block 1 Metadata Linter
//!
//! Evaluates and scores the ontological foundation of `.omni` files:
//! 1. Formal Module Identity (`= [Module]: < ... > & [ ... ]` with intent Aside).
//! 2. Invariant Provenance (`Version`, `Author`, `Created`, `Updated`, `System`).
//! 3. Covenant Witness Grounding (`< Scripture { ... } >` or `[ Witness ]:`).
//! 4. Ontological Invariants & Freestanding Type Definitions (`< Type > = ...`
//!    across the 9 dimensions: Relation, Domain, Substrate, Inscribe, Transduce,
//!    Exemplar, Compare, Sequence, Bound).
//! 5. Spatial Geometry (<= 80 columns) and bracket identifier naming hygiene.

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::MetadataScore;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( ReelMetadataLinter Definition & Slice Extractor )
// ============================================================================

pub struct ReelMetadataLinter;

impl ReelMetadataLinter {
    /// Extracts the text slice corresponding to Block 1 in a `.omni` file.
    pub fn extract_slice(content: &str) -> (usize, usize, String) {
        let lines: Vec<&str> = content.lines().collect();
        if lines.is_empty() {
            return (0, 0, String::new());
        }

        let mut start_idx = None;
        let mut end_idx = None;

        for (idx, line) in lines.iter().enumerate() {
            let upper = line.to_uppercase();
            if start_idx.is_none() {
                if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                    start_idx = Some(idx);
                }
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                end_idx = Some(idx);
                break;
            }
        }

        let start = start_idx.unwrap_or(0);
        let end = end_idx.unwrap_or(lines.len());
        (start + 1, end, lines[start..end].join("\n"))
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Evaluation Engine for Source Reel Metadata Invariants )
// ============================================================================

impl ReelMetadataLinter {
    /// Evaluates and scores Block 1 for a `.omni` Source Reel.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> MetadataScore {
        let mut identity = 0u32;
        let mut provenance = 0u32;
        let mut witness = 0u32;
        let mut ontology = 0u32;
        let mut spatial = 0u32;
        let mut breakdown = Vec::new();

```

```
        // ------------------------------------------------------------------
        // 1. Identity & Formal Declaration (max 20 pts)
        // ------------------------------------------------------------------
        let has_module_decl = slice.contains("= [Module]:")
            || slice.contains("Module <")
            || slice.contains("Module [")
            || slice.contains("Scroll <");
        if has_module_decl {
            identity += 15;
            breakdown.push("✓ Formal Module Identity declared (+15/15)".into());
        } else {
            breakdown.push("✗ Missing formal module identity declaration (0/15)".into());
        }

        let has_identity_aside = slice.lines().any(|l| {
            (l.contains("= [Module]:") || l.contains("Module <")) && l.contains('(')
                || (slice.contains("= [Module]:") && slice.contains("( This is the Module"))
        }) || (has_module_decl && slice.contains('(') && slice.contains(')'));

        if has_identity_aside {
            identity += 5;
            breakdown.push("✓ Module intent Aside '( ... )' attached (+5/5)".into());
        } else {
            breakdown.push("✗ Missing intent Aside on module identity (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 2. Provenance Attributes & Environment (max 25 pts)
        // ------------------------------------------------------------------
        let has_version = slice.contains("[ Version ]") || slice.contains("version:");
        if has_version {
            provenance += 8;
            breakdown.push("✓ Invariant Version attribute specified (+8/8)".into());
        } else {
            breakdown.push("✗ Missing version attribute in metadata context (0/8)".into());
        }

        let has_author = slice.contains("[ Author ]") || slice.contains("author:");
        if has_author {
            provenance += 6;
            breakdown.push("✓ Author / Organization provenance recorded (+6/6)".into());
        } else {
            breakdown.push("✗ Missing author provenance specification (0/6)".into());
        }

        let has_timestamps = slice.contains("[ Created ]") || slice.contains("[ Updated ]");
        if has_timestamps {
            provenance += 6;
            breakdown.push("✓ Created / Updated temporal anchors inscribed (+6/6)".into());
        } else {
            breakdown.push("✗ Missing temporal created/updated timestamps (0/6)".into());
        }

        let has_system = slice.contains("[ System ]") || slice.contains("system:");
        if has_system {
            provenance += 5;
            breakdown.push("✓ Target system / runtime context declared (+5/5)".into());
        } else {
            breakdown.push("✗ Missing target system context declaration (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 3. Covenant & Witness Grounding (max 20 pts)
        // ------------------------------------------------------------------
        let has_scripture = slice.contains("< Scripture") || slice.contains("[ Witness ]");
        if has_scripture {
            witness += 15;
            breakdown.push("✓ Invariant Covenant / Scripture witness anchored (+15/15)".into());
        } else {
            breakdown.push("✗ Missing scripture/covenant witness anchor (0/15)".into());
        }

        let has_witness_aside = slice.contains("Invariant Witness")
            || (has_scripture && slice.contains('(') && slice.contains("Witness"));
        if has_witness_aside {
            witness += 5;
            breakdown.push("✓ Machine-readable Witness Aside oracle attached (+5/5)".into());
        } else if has_scripture {
```

```
            witness += 3;
            breakdown.push("⚠ Living Aside on witness partially detected (+3/5)".into());
        } else {
            breakdown.push("✗ Missing witness Aside oracle (0/5)".into());
        }

        // ---------------------------------------------------------------------
        // 4. Ontological Dimensions & Invariant Types (max 20 pts)
        // ---------------------------------------------------------------------
        let has_freestanding_type = slice.contains("< Character > =")
            || (slice.contains("< ") && slice.contains(" > =") && slice.contains('&'));
        let has_nature_semantics = slice.contains("[ Nature ]") || slice.contains("[ Semantics ]");

        if has_freestanding_type {
            let dimensions = [
                "Relation", "Domain", "Substrate", "Inscribe",
                "Transduce", "Exemplar", "Compare", "Sequence", "Bound",
            ];
            let matched_dims = dimensions.iter().filter(|&&d| slice.contains(d)).count();
            if matched_dims >= 7 {
                ontology += 20;
                breakdown.push(format!(
                    "✓ Freestanding Invariant Type defined across {matched_dims}/9 dimensions (+20/20)"
                ));
            } else {
                let pts = (matched_dims as u32) * 2;
                ontology += pts;
                breakdown.push(format!(
                    "⚠ Freestanding Type defined with partial dimensions ({matched_dims}/9) (+{pts}/20)"
                ));
            }
        } else if has_nature_semantics {
            ontology += 20;
            breakdown.push("✓ Nature and Semantics ontology bounds defined (+20/20)".into());
        } else {
            breakdown.push("✗ Missing freestanding invariant type or nature/semantics bounds (0/20)".into());
        }

        // ---------------------------------------------------------------------
        // 5. Spatial Geometry & Syntax Hygiene within Block 1 (max 15 pts)
        // ---------------------------------------------------------------------
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 1 lines <= {max_cols} columns (+10/10)"));
        } else {
            breakdown.push(format!("✗ Block 1 lines exceed {max_cols}-column spatial ruler (0/10)"));
        }

        let snake_violation = lines.iter().any(|line| {
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                return false;
            }
            if let Some(start) = line.find('[') {
                if let Some(end) = line[start..].find(']') {
                    let inside = &line[start + 1..start + end];
                    for w in inside.split_whitespace() {
                        if w.contains('_') && !w.starts_with("0x") && !w.starts_with("roll_") {
                            return true;
                        }
                    }
                }
            }
            false
        });

        if !snake_violation {
            spatial += 5;
            breakdown.push("✓ Natural spaced identifier hygiene observed (+5/5)".into());
        } else {
            breakdown.push("✗ snake_case identifiers found inside brackets (0/5)".into());
        }

        MetadataScore::new(identity, provenance, witness, ontology, spatial, breakdown)
```

```
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Source Reel Block 1 Linting )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_reel_metadata_extraction_and_empty_check() {
        let (start, end, slice) = ReelMetadataLinter::extract_slice("");
        assert_eq!(start, 0);
        assert_eq!(end, 0);
        assert!(slice.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/score.rs RS | 104 LINES | 452 WORDS | 4.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 1 Metadata & Tape Head Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Metadata & Tape Head Scoring Taxonomy
//!
//! Provides the unified scoring model for Block 1 across both `.omni` Source Reels
//! and `.tape` Process Ribbons. While the concrete syntax of reels and tapes differs,
//! both express the exact same ontological foundation and covenant understanding:
//! 1. Identity & Formal Declaration (max 20 pts)
//! 2. Provenance & Operational Environment (max 25 pts)
//! 3. Covenant & Scripture Invariant Grounding (max 20 pts)
//! 4. Ontological Invariants / Cardinal Coordinates (max 20 pts)
//! 5. Spatial Geometry & Naming Hygiene (max 15 pts)

/// Comprehensive rubric breakdown and total score for Block 1 (0..=100).
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct MetadataScore {
    /// Score for formal identity and module/tape declaration (max 20).
    pub identity_score: u32,
    /// Score for provenance, author, version, and environment (max 25).
    pub provenance_score: u32,
    /// Score for scripture/covenant invariant witness grounding (max 20).
    pub witness_score: u32,
    /// Score for ontological dimensions or 6 cardinal clauses (max 20).
    pub ontology_score: u32,
    /// Score for spatial geometry and delimiter naming hygiene (max 15).
    pub spatial_hygiene_score: u32,
    /// Total aggregate score (0..=100).
    pub total: u32,
    /// Letter grade and descriptive rating.
    pub rating: &'static str,
    /// Granular audit notes explaining points awarded or deducted.
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation & Rubric Evaluation )
// ============================================================================

impl MetadataScore {
    /// Computes the letter grade and descriptive status from the total score.
    pub fn compute_rating(total: u32) -> &'static str {
        match total {
            100 => "A+ (Exemplary Invariant)",
            90..=99 => "A (Compliant)",
            80..=89 => "B (Adequate Foundation)",
            70..=79 => "C (Needs Provenance / Witness Grounding)",
            50..=69 => "D (Deficient Metadata)",
            _ => "F (Unanchored / Incomplete)",
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Summary & Aggregation Methods )
// ============================================================================

impl MetadataScore {
    /// Creates a new MetadataScore from component scores and breakdown notes.
    pub fn new(
        identity: u32,
        provenance: u32,
        witness: u32,
        ontology: u32,
        spatial: u32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = identity + provenance + witness + ontology + spatial;
        Self {
            identity_score: identity,
```

```
            provenance_score: provenance,
            witness_score: witness,
            ontology_score: ontology,
            spatial_hygiene_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for MetadataScore Rating Taxonomy )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_metadata_score_rating_tiers() {
        assert_eq!(MetadataScore::compute_rating(100), "A+ (Exemplary Invariant)");
        assert_eq!(MetadataScore::compute_rating(95), "A (Compliant)");
        assert_eq!(MetadataScore::compute_rating(85), "B (Adequate Foundation)");
        assert_eq!(MetadataScore::compute_rating(75), "C (Needs Provenance / Witness Grounding)");
        assert_eq!(MetadataScore::compute_rating(60), "D (Deficient Metadata)");
        assert_eq!(MetadataScore::compute_rating(30), "F (Unanchored / Incomplete)");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/scaffold/mod.rs

RS | 206 LINES | 1128 WORDS | 7.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Canonical Code Scaffolding Templates for .omni, .tape, and .scroll )
// ============================================================================
//! Canonical Code Scaffolding Engine
//!
//! Generates compliant, 80-column bounded skeleton templates for all OmniCode file formats:
//! - `.omni`: 4-Block Source Reel
//! - `.tape`: Sequential Process Ribbon
//! - `.scroll`: Cognitive Episodic Scroll

use crate::core::filetype::OmniFileType;

pub struct LinterScaffolder;

impl LinterScaffolder {
    /// Auto-scaffolds a compliant 4-Block skeleton for an episodic cognitive scroll (.scroll).
    pub fn auto_scaffold_scroll(session_id: &str, prompt: &str) -> String {
        OmniFileType::EpisodicScroll.scaffold_template(session_id, prompt)
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for a given module name (.omni),
    /// strictly bounded to the 80-column ruler with 3-tier boundary comments.
    pub fn auto_scaffold_omni(module_name: &str, intent_aside: &str) -> String {
        format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea {{ Type }} : {module_name} > )
# ============================================================================
= [Module]: < Lexicon {{ {module_name} }} > & [ Noun {{ {module_name} }} ]
    ( {intent_aside} )
{{
    [ Version ]:   {{ "1.0.0" }}
        ( Version 1.0.0 ),

    [ Author ]:    {{ "CreativeWorkzStudio LLC / ProfessorSeanEX" }}
        ( System Architect ),

    [ System ]:    {{ OmniCode {{ Core }} }}
        ( Core Subsystem ),

    [ Witness ]:   < Scripture {{ Exodus 31:18 KJV }} >
        {{ "Written with the finger of God." }}
        ( Invariant Scripture Anchor ),

    [ Nature ]:    < InvariantType > & {{ BaseBlock }}
        ( Nature is Invariant Type ),

    [ Semantics ]: {{ StandardLibrary :: Specification }}
        ( Semantics specification )
}}

< {module_name} > = < Idea {{ Type }} > & [ Noun {{ {module_name} }} ]
    ( Invariant Definition )
{{
    [ Relation ]: [ {module_name} ] : [ Unit ] :: [ Trit ] : [ Word ]
        ( Ontological Relation ),

    [ Substrate ]: [ 1 Byte {{ 8 Bits }} ] & [ 1 Cell {{ 5 Trits }} ]
        ( Dual Substrate ),

    [ Bound ]: {{ {module_name} }} = [ Enclosed ]: TRUE
        ( Invariant Single Cell Boundary )
}}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {{
    [ Entity ]: < {module_name} > & [ Cell ]
        ( Entity allocated in cell ),

    [ Cell ]: [ 1 ] & {{ 5 Trits :: 1 Byte }}
```

```
        ( One cell container ),

    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Equilibrium verified )
}}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    [ ROLL 1 ] : [ Inscribe ] :
        [ Input ] --> INSCRIBE [ Cell ] = [ Entity` ]
        ( Inscribe input into Cell yielding Entity-prime ),

    [ ROLL 2 ] : [ Compare ] :
        {{ Entity` = Input }} --> [ Status` = +1:PROCEED ]
        | {{ Entity` != Input }} --> [ Status` = -1:ROLLBACK ]
        ( Evaluate identity and verify bounds )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing
    ( Sealing Gate: Validate, tripwire, debug, log, and affirm execution )
{{
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED
    ]
    ( Verify that all rolls achieved affirmative PROCEED ),

    [ Validate ]:
        {{ Entity` <= [ 1 Cell ] }} & {{ [ 8 Bits ] <=> [ 5 Trits ] }}
        ( Validate single cell bounds and substrate parity ),

    [ Tripwire ]:
        {{ Status` = +1:PROCEED }} --> [ ARMED ]
        | {{ Status` != +1:PROCEED }} --> [ TRIPPED ]
            --> [ Commit Status = -1:ROLLBACK ]
        ( Circuit breaker vetoes on anomaly ),

    [ Debug ]:
        {{ [ Entity` ] :: [ Cell` ] :: [ Status` ] }}
        ( Expose diagnostic telemetry snapshot ),

    [ Log ]: [ Journal ]:
        INSCRIBE {{ [ {module_name} Verified ] :: [ Entity` ] }}
        ( Inscribe event record into system journal ),

    [ Commit Status ]: [
        {{ [ Verify ] = PROCEED }} & {{ [ Tripwire ] = ARMED }}
            --> [ +1:PROCEED ] |
        {{ [ Status` ] = PENDING }}
            --> [ 0:AWAIT ] :: REPEAT |
        {{ [ Status` ] = NULL }}
            --> [ HALT:NULL ] |
        {{ [ Tripwire ] = TRIPPED }} | {{ [ Verify ] != PROCEED }}
            --> [ -1:ROLLBACK ]
    ]
    ( Fourfold state machine: Proceed, Await, Halt, Rollback )
}}
"#
        )
    }

    /// Auto-scaffolds a compliant `.tape` sequential process ribbon skeleton,
    /// strictly bounded to 80 columns with Tape Head and Cardinal Clauses.
    pub fn auto_scaffold_tape(tape_name: &str, intent_aside: &str) -> String {
        format!(
r#"# ================================================================================
# PROCESS TAPE: {upper} INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Type < {tape_name} >.
#   - Defined Space: [ 1 Cell ] {{ 5 Trits :: 1 Byte }}
#   - Execution Domain: Lexicon {{ Types of Ideas }}
#   - Witness: Scripture Exodus 31:18 KJV
```

```
# ================================================================================

= [Tape]: < Lexicon {{ Types of Ideas }} > & [ Process {{ {tape_name} }} ] {{
    WHO   {{ OmniCode Lexicon }} ::
    WHAT  {{ {tape_name} }} --> DISPATCH {{ Process }} ::
    WHEN  {{ Input Event }} ::
    WHERE {{ {tape_name} Cell }} = [ 1 ] ::
    WHY   ( {intent_aside} ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture {{ Exodus 31:18 KJV }} > ::
    :: {{ "Written with the finger of God." }} ::
    :: ( Invariant Witness Exodus 31:18: Written with finger of God )
}}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: INITIAL STATE ALLOCATION
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] :
    [ Input ] --> INSCRIBE [ Cell ] = [ {tape_name}` ]
    ( Inscribe initial input into cell container )

# --------------------------------------------------------------------------------
# ROLL 2: INVARIANT VERIFICATION
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Compare ] :
    {{ {tape_name}` = Input }} ! GUARD --> [ Status` = +1:PROCEED ]
    | {{ {tape_name}` != Input }} --> [ Status` = -1:ROLLBACK ]
    ( Verify invariant bounds under guard )

# --------------------------------------------------------------------------------
# ROLL 3: COMMIT STATUS & FOURFOLD STATE MACHINE
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Commit Status ] : [
    {{ [ Verify ] = PROCEED }} & {{ [ Tripwire ] = ARMED }}
        --> [ +1:PROCEED ] |
    {{ [ Status` ] = PENDING }}
        --> [ 0:AWAIT ] :: REPEAT |
    {{ [ Status` ] = NULL }}
        --> [ HALT:NULL ] |
    {{ [ Tripwire ] = TRIPPED }} | {{ [ Verify ] != PROCEED }}
        --> [ -1:ROLLBACK ]
]
( Fourfold commit status: Proceed, Await, Halt, Rollback )
"#,
            upper = tape_name.to_uppercase(),
            tape_name = tape_name,
            intent_aside = intent_aside
        )
    }
}
```